Governance With TERM Token Veto Configuration Proof

1. High Level Conclusion

```
Branch 1 (the slow path, for real proposals)
  ProposerSafe (5/11) ------module------> Delay Modifier ------module------> OwnerlessSafe (9/9) ------DEVOPS_ROLE------> Term Protocol
  (+ ConfigLockGuard)                     (+ PauseGuard)                  (+ ConfigLockGuard)

Direct path (all nine OwnerlessSafe signers together: no delay, veto or pause)
  OwnerlessSafe (9/9) ------DEVOPS_ROLE------> Term Protocol

Branch 2 (the token-vote path, whose only power is to veto)
  Governor ------module------> Roles Modifier ------target------> DelayOwnerSafe ------owner------> Delay Modifier
                           (+ SetTxNonceGuard)                (+ ConfigLockGuard)

Pause controls
  PauseSafe (2/9) --pause-->               PauseGuard
  ProposerSafe   --unpause / setPauser-->  PauseGuard
```

Trust assumption: Despite its name, the OwnerlessSafe has nine Term multisig holders as owners and a threshold of 9, high enough that its direct path has more friction than going through the Delay Modifier.

The conclusions drawn from these proofs establish the following propositions:

1. The Term multisig holders on the ProposerSafe and the Term multisig holders on the OwnerlessSafe, and only they, may execute DEVOPS_ROLE methods, holding: 1. the owners, fallbackHandler, modules, and guard constant for the ProposerSafe and OwnerlessSafe; 2. the module guard constant for the OwnerlessSafe; 3. the owner, target, modules, guard, txCooldown, and txExpiration constant for the Delay Modifier; and 4. the pauser and admin constant for the PauseGuard, provided DEVOPS_ROLE method calls from the ProposerSafe are not vetoed, and are executed while the PauseGuard is not paused and before the 24-hour expiration period following the cooldown ends.
2. Only TERM token holders, through a proposal made by a holder with at least 1,000 TERM of voting power that reaches a quorum of 1,000,000 TERM, if proposed within 2 hours of the transaction being queued, and Term multisig holders may veto Delay Modifier transactions within the 24-hour veto window, in which a Delay Modifier transaction cannot be executed until txCooldown has passed since it was queued, holding: 1. the owners, fallbackHandler, modules, guard, module guard constant for the DelayOwnerSafe, and 2. the owner, target, modules, guard constant for the Delay Modifier and Roles Modifier, and 3. the role targets, role scopes, role assignments constant for the Roles Modifier, and 4. the voting token, proposal threshold, voting threshold, quorum constant for the Governor, and 5. the txCooldown constant for the Delay Modifier.
3. Only the Term multisig holders on the PauseSafe may pause, and only the Term multisig holders on the ProposerSafe may unpause, the execution of Delay Modifier transactions, and if a pause outlasts the 24-hour expiration period following the cooldown, the Delay Modifier transaction can only be skipped or vetoed, holding: 1. the owners, fallbackHandler, modules, guard constant for the PauseSafe and ProposerSafe, and 2. the guard, txCooldown, txExpiration constant for the Delay Modifier, and 3. the pauser and admin constant for the PauseGuard.

4. The following: 1. the owner, target, modules, guard, role targets, role scopes, role assignments for the Roles Modifier, and 2. the voting token, proposal threshold, voting threshold, quorum for the Governor cannot be changed by anyone.
5. The owners for the ProposerSafe, OwnerlessSafe, and DelayOwnerSafe can be changed only by the Term multisig holders on the ProposerSafe, OwnerlessSafe, or DelayOwnerSafe. The fallbackHandler, modules, guard, module guard for the OwnerlessSafe can be changed only by the Term multisig holders on the ProposerSafe. The following: 1. the fallbackHandler, modules, guard for the ProposerSafe and DelayOwnerSafe, 2. the module guard for the DelayOwnerSafe, and 3. the owner, target, modules, guard, txCooldown, txExpiration for the Delay Modifier cannot be changed by anyone.
6. The owners, fallbackHandler, modules, guard for the PauseSafe can be changed only by the Term multisig holders on the PauseSafe. The pauser for the PauseGuard can be changed only by the Term multisig holders on the ProposerSafe. The admin for the PauseGuard cannot be changed by anyone.

Only Term multisig holders can execute DEVOPS_ROLE methods subject to TERM token governance, and this cannot be changed other than through Term multisig approval.

2. Evidence Classification

This document uses two classes of evidence to establish the security and functional properties of the governance configuration:

- **Formal Verification (FV) evidence** establishes that, assuming the stated configuration, the relevant contracts enforce the claimed security and functional properties. These properties are proved with the Certora Prover, a formal verification tool. For the guards (PauseGuard, ConfigLockGuard and SetTxNonceGuard), the proofs run on the contract code that was deployed, the code at tag `2.1.0`. For TERM, the Governor rules that read TERM run on the code the TERM proxy is to be upgraded to, the code at commit `fa1722faf0bc0e953efc65394e35e7919cb2793d` on term-token-contracts `main`. For the Governor, the proofs run on term-finance-ops-contracts' `TermFinanceProtocolGovernor.sol` at commit `5d90281be203389bdc465963164f635048e5f3e6`, the head of PR 1096, built with OpenZeppelin 5.6.1. The proofs cover privileged entry points, authorized callers, configuration immutability, the absence of alternative execution paths, and the intended functionality. In the tables below, FV evidence is labeled **FV —** and names each Certora rule; ✅ marks a rule that passes.
- **Deployment evidence** states the configuration the 2.1.0 deployment must have for the formal verification evidence to apply: owners, modules, guards, thresholds, roles, and other relevant state. 2.1.0 is a proposal, not a live deployment, so none of it has been read on chain yet. In the tables below, deployment evidence is labeled **On-chain —**; ⏳ marks a requirement to verify against on-chain state, at a stated block, once the 2.1.0 contracts are deployed.

These two classes of evidence are complementary. Formal verification evidence does not by itself establish that the deployed contracts have the configuration being proved, and deployment evidence does not by itself establish that the deployed configuration enforces the claimed security and functional properties.

Safe versions. This document assumes every Safe in this configuration, the ProposerSafe, OwnerlessSafe, DelayOwnerSafe and PauseSafe, runs Safe v1.5.0, the non-L2 build. The formal verification evidence covers Safe v1.5.0 only.

Configuration items marked ⏳ (pending deployment) describe the configuration of the proposed 2.1.0 deployment and are not established yet. Each must be verified against on-chain state once the 2.1.0 contracts are deployed.

3. Evidence

Proposition 1 — The Term multisig holders on the ProposerSafe and the Term multisig holders on the OwnerlessSafe, and only they, may execute DEVOPS_ROLE methods, holding: 1. the owners, fallbackHandler, modules, and guard constant for the ProposerSafe and OwnerlessSafe; 2. the module guard constant for the OwnerlessSafe; 3. the owner, target, modules, guard, txCooldown, and txExpiration constant for the Delay Modifier; and 4. the pauser and admin constant for the PauseGuard, provided DEVOPS_ROLE method calls from the ProposerSafe are not vetoed, and are executed while the PauseGuard is not paused and before the 24-hour expiration period following the cooldown ends.

#	Statement	Evidence
P1.1	Only the OwnerlessSafe may execute DEVOPS_ROLE methods.	**On-chain — The OwnerlessSafe is the only holder of the DEVOPS_ROLE AccessControl role required for DEVOPS_ROLE methods:** • The DEVOPS_ROLE methods are those listed in the DEVOPS_ROLE row of Term Finance's Access Controls page: `_authorizeUpgrade` on all contracts, `pauseDeploying` and `unpauseDeploying` on TermInitializer, `updateTreasuryAddress` and `updateProtocolReserveAddress` on TermController, and `addNewTokenPriceFeed` and `removeTokenPriceFeed` on TermPriceConsumerV3. • ⏳ pending deployment. The six Term Protocol contracts (TermController, TermEventEmitter, TermPriceConsumerV3, TermRepoDeployerFactory, TermInitializer and TermDiamond) are redeployed alongside the 2.1.0 guards, each granting `DEVOPS_ROLE` to the OwnerlessSafe and to no other address: in each new contract's log history, the only `RoleGranted` naming `DEVOPS_ROLE` must be for the OwnerlessSafe, with no `RoleRevoked` and no `RoleAdminChanged`. • The other contracts with `_authorizeUpgrade` are proxies deployed against implementations, such as each term's TermAuction. Term's deploy system deploys every new proxy with `DEVOPS_ROLE` assigned only to the OwnerlessSafe. The deploy system runs off chain, so this is a deployment fact, not an on-chain read. The existing terms' repo and auction proxies do not assign `DEVOPS_ROLE` to the OwnerlessSafe, so new ones need to be deployed alongside the 2.1.0 guards. **On-chain — Nobody can grant DEVOPS_ROLE to another address that is not the OwnerlessSafe:** • ⏳ pending deployment. On TermController, TermEventEmitter, TermPriceConsumerV3, TermInitializer and TermRepoDeployerFactory, `getRoleAdmin(DEVOPS_ROLE)` must be `DEFAULT_ADMIN_ROLE` (`0x00`), so only a holder of `DEFAULT_ADMIN_ROLE` can call `grantRole(DEVOPS_ROLE, …)`. • ⏳ pending deployment. No address may hold `DEFAULT_ADMIN_ROLE` on any of the six contracts: their log histories must have no `RoleGranted` for it. So nobody can grant `DEVOPS_ROLE` on those five contracts.
P1.2	Only Modules of OwnerlessSafe and Owners of OwnerlessSafe can interact with the OwnerlessSafe, except `execTransaction`, which anyone can call only if Owners of OwnerlessSafe have approved the transaction, and except `fallback` and `receive`, which anyone can call but which do not change state, assuming the fallbackHandler is `address(0)`.	**FV — Callers can only interact with the Safe through `execTransactionFromModule`, `execTransactionFromModuleReturnData`, `execTransaction`, `approveHash`, `enableModule`, `disableModule`, `addOwnerWithThreshold`, `removeOwner`, `swapOwner`, `changeThreshold`, `setGuard`, `setFallbackHandler`, `setModuleGuard`, `setup`, `simulateAndRevert`, `fallback` and `receive`:** • SE150-1 `safeEntryPointsAreAllAccountedFor` ✅ passes: Safe v1.5.0 has no write functions besides `execTransactionFromModule`, `execTransactionFromModuleReturnData`, `execTransaction`, `approveHash`, `enableModule`, `disableModule`, `addOwnerWithThreshold`, `removeOwner`, `swapOwner`, `changeThreshold`, `setGuard`, `setFallbackHandler`, `setModuleGuard`, `setup`, `simulateAndRevert`, `fallback` and `receive` (the Prover checks `fallback` and `receive` as one entry), and any unsupported call lands in `fallback`. Each blob below covers some of these entry points, and together they cover all seventeen. **FV — `execTransactionFromModule` and `execTransactionFromModuleReturnData` are only callable by Modules of**

#	Statement	Evidence
		Safe: • SE150-3 `onlyEnabledModulesCanCallModuleExec` ✅ passes: only an enabled module (Modules of Safe) can successfully call `execTransactionFromModule` or `execTransactionFromModuleReturnData`. • SE150-4 `enabledModuleCanMakeTheSafeAct` ✅ passes: an enabled module can call them directly, with no owner signatures, and the Safe makes an outgoing call or delegatecall. So SE150-3 is not vacuous. **FV — `approveHash` is only callable by Owners of Safe:** • SE150-8 `onlyOwnersCanApproveHashes` ✅ passes: in any call, an approval is only recorded when the caller is an owner (Owners of Safe) recording its own approval. So a call to `approveHash` from anyone else records nothing. An owner's recorded approval then counts as that owner's signature in `execTransaction`. **FV — `execTransaction` is callable by anyone, but only executes once a threshold of Owners of Safe have approved:** • SE150-5 `execTransactionRunsOnlyAfterTheSignatureCheck` ✅ passes: whoever the caller is, `execTransaction` only runs once the signature check has passed for that exact transaction, asking for as many signatures as the threshold. The rule places no restriction on the caller. • SE150-12 `eachSignatureAcceptsANewApprovingOwner` ✅ passes: each signature the check accepts comes from a different owner (Owners of Safe) who approved the transaction. For an approval made through `approveHash`, SE150-8 ensures the owner recorded it itself. So with SE150-5, the Safe acts through `execTransaction` only once as many different owners as the threshold have approved. • SE150-6 `ownersCanMakeTheSafeAct` ✅ passes: once the threshold is passed, `execTransaction` succeeds and the Safe acts, for any threshold and any number of owners (Owners of Safe). So SE150-5 is not vacuous. **FV — `enableModule`, `disableModule`, `addOwnerWithThreshold`, `removeOwner`, `swapOwner`, `changeThreshold`, `setGuard`, `setFallbackHandler` and `setModuleGuard` are only callable from Modules of Safe or Owners of Safe:** • SE150-7 `settingsOnlyChangeWhenTheSafeCallsItself` ✅ passes: in any call, the owners, threshold, modules, guard, module guard and fallbackHandler only change when the caller is the Safe itself. So a call to any of these nine functions from anyone else changes nothing. • SC150-4 `settingsOnlyChangeThroughAModuleOrOwners` ✅ passes: over every write function, `fallback` included while no fallbackHandler is set (P1.3), a setting only changes through an enabled module's (Modules of Safe) `execTransactionFromModule` or `execTransactionFromModuleReturnData`, or through `execTransaction`. By the `execTransactionFromModule` and `execTransaction` blobs above, those paths are open only to Modules of Safe and to a threshold of Owners of Safe. • SC150-1 `moduleCanChangeSettings`, SC150-2 `moduleCanChangeSettingsWithReturnData` ✅ pass: for each function above, a module's (Modules of Safe) `execTransactionFromModule`, and likewise its `execTransactionFromModuleReturnData`, addressed to the Safe with a call to that function, succeeds and changes a setting. • SC150-3 `ownersCanChangeSettings` ✅ passes: for each function above, an owner's (Owners of Safe) `execTransaction`, addressed to the Safe with a call to that function, succeeds and changes a setting, once the threshold is passed. Together with SC150-1 and SC150-2, this shows the restriction is not achieved by nothing working. **FV — `setup` always reverts after deploy:** • SE150-9 `setupAlwaysRevertsAfterSetup` ✅ passes: once the Safe is set up, `setup` always reverts, so nobody can re-run it to replace the owners, threshold, modules or handler. • The OwnerlessSafe is set up: its threshold is 9 and it has nine owners (P1.4). **FV — `simulateAndRevert` always reverts:** • SE150-11 `simulateAndRevertAlwaysReverts` ✅ passes: anyone can call `simulateAndRevert`, and it does make a

#	Statement	Evidence
		delegatecall, but it always reverts afterwards, so none of its effects persist. **FV — `fallback` is callable by anyone but does not change state, assuming the fallbackHandler is `address(0)`:** • SE150-21 `noHandlerFallbackChangesNoState` ✅ passes: with no handler set, `fallback` changes no state at all. It writes no storage, makes no call of any kind (call, delegatecall, staticcall or callcode) and creates or destroys no contract, and a `fallback` call that succeeds carries no ETH, so not even the Safe's balance changes. • SE150-23 `storageHookFires`, SE150-24 `callHookFires` ✅ pass: a successful `approveHash` is seen writing the Safe's storage, and `fallback` with a handler set is seen making its call. So SE150-21's checks do detect storage writes and calls. • SE150-22 `noHandlerFallbackSucceeds` ✅ passes: with no handler set, some call to `fallback` succeeds, so SE150-21 is not vacuous. **FV — `receive` is callable by anyone but does not change state:** • SE150-25 `receiveChangesNoState` ✅ passes: whatever the handler, a call that runs `receive` changes no state beyond the ETH it is sent. It writes no storage, makes no call of any kind and creates or destroys no contract. • Here, state means the Safe's storage, which holds its owners, threshold, modules, guard, module guard, fallbackHandler, nonce and approvals. The ETH sent is added to the Safe's balance, which is not part of that storage. • SE150-23 `storageHookFires`, SE150-24 `callHookFires` ✅ pass: a successful `approveHash` is seen writing the Safe's storage, and `fallback` with a handler set is seen making its call. So SE150-25's checks do detect storage writes and calls. • SE150-26 `receiveRuns` ✅ passes: some call to the Safe runs `receive` and succeeds, so SE150-25 is not vacuous. **On-chain — The OwnerlessSafe runs Safe v1.5.0, the code the SE150 and SC150 rules check:** • ⏳ pending deployment. The OwnerlessSafe must be a Safe v1.5.0 proxy: `VERSION()` must return `1.5.0`, and storage slot 0 (the proxy's `singleton`) must hold the canonical Safe v1.5.0 singleton. `VERSION()` alone does not tell Safe v1.5.0 from SafeL2 v1.5.0; the singleton does. • So the rules above, which check Safe v1.5.0, hold for the OwnerlessSafe. • Assumes the OwnerlessSafe runs Safe v1.5.0, the version SE150 and SC150 check.
P1.3	OwnerlessSafe fallbackHandler is `address(0)`, holding: 1. the owners, fallbackHandler, modules, and guard constant for the ProposerSafe and OwnerlessSafe; 2. the module guard constant for the OwnerlessSafe; 3. the owner, target, modules, guard, txCooldown, and txExpiration constant for the Delay Modifier; and 4. the pauser and admin constant for the PauseGuard.	**On-chain — OwnerlessSafe fallbackHandler is `address(0)`:** • ⏳ pending deployment. The fallbackHandler slot (`0x6c9a6c4a39284e37ed1cf53d337577d14212a4870fb976a4366c693b939918d5`, `FallbackManager.sol:13` in Safe v1.5.0) on the OwnerlessSafe must hold `address(0)`. • Assumes the OwnerlessSafe runs Safe v1.5.0 (`FallbackManager.sol:13`).
P1.4	`Owners of OwnerlessSafe = Term multisig holders on OwnerlessSafe`, holding: 1. the owners, fallbackHandler, modules, and guard constant for the ProposerSafe and OwnerlessSafe; 2. the module guard constant for the OwnerlessSafe; 3. the owner, target, modules, guard, txCooldown, and txExpiration constant for the Delay Modifier;	**On-chain — Owners of OwnerlessSafe are nine Term multisig holders, and all nine must approve each transaction:** • ⏳ pending deployment. The OwnerlessSafe must run Safe v1.5.0 (`VERSION()`), with threshold 9 (`getThreshold`) and nine owners (`getOwners`), each an externally owned account, with no contract code, held by a Term multisig holder. • Assumes the OwnerlessSafe runs Safe v1.5.0. • The identities of the nine key holders are not visible on chain. Their status as Term multisig holders is a deployment fact.

#	Statement	Evidence
	and 4. the pauser and admin constant for the PauseGuard.	
P1.5	`Modules of OwnerlessSafe = Delay Modifier`, holding: 1. the owners, fallbackHandler, modules, and guard constant for the ProposerSafe and OwnerlessSafe; 2. the module guard constant for the OwnerlessSafe; 3. the owner, target, modules, guard, txCooldown, and txExpiration constant for the Delay Modifier; and 4. the pauser and admin constant for the PauseGuard.	**On-chain — The Modules of OwnerlessSafe list contains only the Delay Modifier:** • ⏳ pending deployment. `getModulesPaginated` on the OwnerlessSafe must return just the Delay Modifier, and report that the list ends there. • Assumes the OwnerlessSafe runs Safe v1.5.0.
P1.6	Only the Modules of the Delay Modifier and the Owner of the Delay Modifier can interact with the Delay Modifier, except `executeNextTx` and `skipExpired`, which anyone can call, but they can only execute or skip transactions queued by the Modules of the Delay Modifier or the Owner of the Delay Modifier.	**FV — Callers can only interact with the Delay Modifier through `setTxCooldown`, `setTxExpiration`, `setTxNonce`, `setAvatar`, `setTarget`, `enableModule`, `disableModule`, `setGuard`, `transferOwnership`, `renounceOwnership`, `execTransactionFromModule`, `execTransactionFromModuleReturnData`, `executeNextTx`, `skipExpired` and `setUp`:** • DM-13 `delayWriteFunctionsAreTheKnownFifteen` ✅ passes: the Delay Modifier has no fallback and no write functions besides `setTxCooldown`, `setTxExpiration`, `setTxNonce`, `setAvatar`, `setTarget`, `enableModule`, `disableModule`, `setGuard`, `transferOwnership`, `renounceOwnership`, `execTransactionFromModule`, `execTransactionFromModuleReturnData`, `executeNextTx`, `skipExpired` and `setUp`. Each blob below covers some of these, and together they cover all fifteen. **FV — `execTransactionFromModule` and `execTransactionFromModuleReturnData` are only callable by Modules of Delay Modifier:** • DM-20 `onlyEnabledModulesCanCallExecFromModule` ✅ passes: only an enabled module (Modules of Delay Modifier) can successfully call `execTransactionFromModule` or `execTransactionFromModuleReturnData`. • DM-5 `enabledModuleCanQueue` ✅ passes: an enabled module's call to `execTransactionFromModule` succeeds. So DM-20 is not vacuous. **FV — `setTxCooldown`, `setTxExpiration`, `setTxNonce`, `setAvatar`, `setTarget`, `enableModule`, `disableModule`, `setGuard`, `transferOwnership` and `renounceOwnership` are only callable by Owner of Delay Modifier:** • DM-8 `atMostOneCallerPassesOnlyOwner` ✅ passes: each of these ten functions succeeds only when the caller is the owner (Owner of Delay Modifier), and from the same state no two different callers can both succeed. • DM-19 `ownerCanCallEachDelaySetting` ✅ passes: for each of the ten, some call from the owner succeeds. So DM-8 is not vacuous. **FV — `setUp` always reverts after deploy:** • DM-1 `setUpAlwaysRevertsAfterDeployment` ✅ passes: nobody can re-run the Delay Modifier's setup after deployment. **FV — `executeNextTx` and `skipExpired` are callable by anyone, but can only execute or skip transactions queued by Modules of Delay Modifier or Owner of Delay Modifier:** • DM-11 `anyoneCanExecuteNextTx`, DM-12 `anyoneCanSkipExpired` ✅ pass: whether a call to `executeNextTx` or `skipExpired` succeeds, and what it changes, does not depend on who the caller is. • DM-17 `outsiderCanExecuteNextTx`, DM-18 `outsiderCanSkipExpired` ✅ pass: a caller that is neither the owner nor a module can execute or skip a queue entry. So DM-11 and DM-12 are not vacuous.

#	Statement	Evidence
		• DQ-2 `everyQueuedEntryWasQueuedByModuleOrOwner` ✅ passes: every transaction in the queue was queued by Modules of Delay Modifier or Owner of Delay Modifier. • DQ-3 `executeNextTxRunsOnlyEntriesQueuedByModuleOrOwner` ✅ passes: `executeNextTx` only runs the queue entry at `txNonce`. By DQ-2, that entry was queued by Modules of Delay Modifier or Owner of Delay Modifier. • DQ-4 `skipExpiredSkipsOnlyEntriesQueuedByModuleOrOwner` ✅ passes: every index `skipExpired` steps over is a queue entry, and by DQ-2 it was queued by Modules of Delay Modifier or Owner of Delay Modifier. • DQ-5 `executeNextTxCanRunAQueuedEntry`, DQ-6 `skipExpiredCanSkipAQueuedEntry` ✅ pass: `executeNextTx` can run a queue entry, and `skipExpired` can skip one. So DQ-3 and DQ-4 are not vacuous.
P1.7	Only the Modules of the Delay Modifier can queue transactions to the Delay Modifier. The Owner of the Delay Modifier, the DelayOwnerSafe, cannot queue transactions to the Delay Modifier.	**FV — Only Modules of Delay Modifier can queue transactions to the Delay Modifier:** • DM-4 `queueOnlyGrowsThroughEnabledModules` ✅ passes: over every write function, only an enabled module (Modules of Delay Modifier) can move the queue nonce or write a queue entry. • DM-5 `enabledModuleCanQueue` ✅ passes: an enabled module's call to `execTransactionFromModule` succeeds and moves the queue nonce forward by one. So DM-4 is not vacuous. **On-chain — Owner of Delay Modifier, the DelayOwnerSafe, is not a Module of Delay Modifier:** • ⏳ pending deployment. `owner()` on the Delay Modifier must return the DelayOwnerSafe (P2.3). • ⏳ pending deployment. `isModuleEnabled` on the Delay Modifier must return false for the DelayOwnerSafe. This matches P1.8: the Delay Modifier's only module (Modules of Delay Modifier) is the ProposerSafe. • So, since the DelayOwnerSafe is the owner, by DM-4 the Owner of Delay Modifier cannot queue transactions.
P1.8	`Modules of Delay Modifier = ProposerSafe`, holding: 1. the owners, fallbackHandler, modules, and guard constant for the ProposerSafe and OwnerlessSafe; 2. the module guard constant for the OwnerlessSafe; 3. the owner, target, modules, guard, txCooldown, and txExpiration constant for the Delay Modifier; and 4. the pauser and admin constant for the PauseGuard.	**On-chain — The Modules of Delay Modifier list contains only the ProposerSafe:** • ⏳ pending deployment. `getModulesPaginated` on the Delay Modifier must return just the ProposerSafe, and report that the list ends there.
P1.9	Only Modules of ProposerSafe and Owners of ProposerSafe can interact with the ProposerSafe, except `execTransaction`, which anyone can call only if Owners of ProposerSafe have approved the transaction, and except `fallback` and `receive`, which anyone can call but which do not change state, assuming the fallbackHandler is `address(0)`.	Same as P1.2. **On-chain — The ProposerSafe runs the same Safe v1.5.0 code as the OwnerlessSafe, so P1.2's evidence applies to the ProposerSafe:** • ⏳ pending deployment. The ProposerSafe must be a Safe v1.5.0 proxy: `VERSION()` must return `1.5.0`, and storage slot 0 (the proxy's `singleton`) must hold the canonical Safe v1.5.0 singleton. • ⏳ pending deployment. The ProposerSafe and the OwnerlessSafe must have the same runtime code, the Safe v1.5.0 proxy's. • So both Safes run the same proxy in front of the same mastercopy, and P1.2's rules, which check Safe v1.5.0, hold for the ProposerSafe too. • Assumes the ProposerSafe runs Safe v1.5.0, the version SE150 and SC150 check.
P1.10	ProposerSafe fallbackHandler is `address(0)`, holding: 1. the owners, fallbackHandler, modules, and guard constant for the ProposerSafe and	**On-chain — ProposerSafe fallbackHandler is `address(0)`:** • ⏳ pending deployment. The fallbackHandler slot (`0x6c9a6c4a39284e37ed1cf53d337577d14212a4870fb976a4366c693b939918d5`, `FallbackManager.sol:13` in Safe

#	Statement	Evidence
	OwnerlessSafe; 2. the module guard constant for the OwnerlessSafe; 3. the owner, target, modules, guard, txCooldown, and txExpiration constant for the Delay Modifier; and 4. the pauser and admin constant for the PauseGuard.	v1.5.0) on the ProposerSafe must hold `address(0)`. • Assumes the ProposerSafe runs Safe v1.5.0 (`FallbackManager.sol:13`).
P1.11	`Owners of ProposerSafe = Term multisig holders on ProposerSafe`, holding: 1. the owners, fallbackHandler, modules, and guard constant for the ProposerSafe and OwnerlessSafe; 2. the module guard constant for the OwnerlessSafe; 3. the owner, target, modules, guard, txCooldown, and txExpiration constant for the Delay Modifier; and 4. the pauser and admin constant for the PauseGuard.	**On-chain — Owners of ProposerSafe are eleven Term multisig holders, and five of them must approve each transaction:** • ⏳ pending deployment. The ProposerSafe must run Safe v1.5.0 (`VERSION()`), with threshold 5 (`getThreshold`) and eleven owners (`getOwners`), each an externally owned account, with no contract code, held by a Term multisig holder. • Assumes the ProposerSafe runs Safe v1.5.0. • The identities of the eleven key holders are not visible on chain. Their status as Term multisig holders is a deployment fact.
P1.12	`Modules of ProposerSafe = null`, holding: 1. the owners, fallbackHandler, modules, and guard constant for the ProposerSafe and OwnerlessSafe; 2. the module guard constant for the OwnerlessSafe; 3. the owner, target, modules, guard, txCooldown, and txExpiration constant for the Delay Modifier; and 4. the pauser and admin constant for the PauseGuard.	**On-chain — The Modules of ProposerSafe list is empty:** • ⏳ pending deployment. `getModulesPaginated` on the ProposerSafe must return an empty list. • Assumes the ProposerSafe runs Safe v1.5.0.
P1.13	The Term multisig holders on the ProposerSafe and the Term multisig holders on the OwnerlessSafe may execute DEVOPS_ROLE methods, provided DEVOPS_ROLE method calls from the ProposerSafe are not vetoed, and are executed while the PauseGuard is not paused and before the 24-hour expiration period following the cooldown ends.	**FV — The Term multisig holders on the ProposerSafe can execute DEVOPS_ROLE methods by proposing them to the Delay Modifier and executing them after the cooldown, provided they are not vetoed, while the PauseGuard is not paused and before the proposal expires:** • DE150-1 `proposerOwnersExecuteDevopsMethodThroughTheDelay` ✅ passes. In one execution: ◦ An owner of the ProposerSafe (a Term multisig holder, P1.11) approves with `approveHash` the ProposerSafe's call to the Delay Modifier's `execTransactionFromModule(<protocol contract>, 0, <DEVOPS_ROLE method call>, Call)`. ◦ Anyone submits it through `execTransaction` with that approval as the signature, so the Delay Modifier queues the proposal. ◦ Nobody vetoes it. Once the cooldown has passed, before the proposal expires, and with the PauseGuard installed and not paused, anyone's `executeNextTx` hands it to the OwnerlessSafe. ◦ The OwnerlessSafe calls the protocol contract. The call passes `onlyRole(DEVOPS_ROLE)` because the OwnerlessSafe holds `DEVOPS_ROLE` (P1.1), and the method runs. • DE150-1 sets the ProposerSafe's threshold to 1, so one owner's approval is the whole threshold. SE150-6 `ownersCanMakeTheSafeAct` (P1.2) ✅ passes for any threshold, so with the real threshold of 5 (P1.11), five owners' approvals take the same path. • DE150-1 assumes the deployed wiring: the ProposerSafe is a module of the Delay Modifier (P1.8), and the Delay Modifier is a module of the OwnerlessSafe (P1.5). The Delay Modifier's target is the OwnerlessSafe and its guard is the PauseGuard. DE150-1 also assumes neither Safe has a guard; each has a ConfigLockGuard, which the next two bullets cover. The on-

#	Statement	Evidence
		chain blob below covers the parts not shown elsewhere. • The ProposerSafe's ConfigLockGuard lets the proposal through. Its `lockedDelay` is the Delay Modifier (P5.DelayModifier.13), so the proposal, `execTransactionFromModule` on the Delay Modifier, is a plain call to the locked modifier. It is not one of the nine calls the guard locks there (CL150-6 to CL150-9, CL150-24). CL150-10 `checkTransactionAcceptsEverythingElse` ✅ passes: whatever `lockedDelay` is, the Delay Modifier included, the guard accepts every plain call that is not a locked call. It checks ConfigLockGuard's `checkTransaction` directly, so the Safe version does not matter. Its `checkAfterExecution` does nothing (`ConfigLockGuard.sol:141`), so DE150-1's ProposerSafe step is unchanged. • The OwnerlessSafe's ConfigLockGuard never checks the Delay Modifier's call. Safe v1.5.0 consults its guard only in `execTransaction` (`Safe.sol:149–204`), not in `execTransactionFromModule` (`ModuleManager.sol:154–163`, which consults only the module guard, `ModuleManager.sol:94–109`). CL150-22 `moduleCallBypassesTheGuard` ✅ passes: with ConfigLockGuard installed, an enabled module's `execTransactionFromModule` addressed to the Safe with a `setGuard` call succeeds and moves the guard out of its slot. The guard rejects that call on the signed path (CL150-12), so it is not consulted on the module path. LG150-3 `moduleCanStillCallOut` ✅ passes: with ConfigLockGuard installed and its `lockedDelay` the Delay Modifier (P5.DelayModifier.13), a plain `execTransactionFromModule` from the Delay Modifier, as an enabled module, to another contract succeeds. • DE150-1 also assumes the PauseGuard is not paused. While it is paused, the PauseGuard rejects every `executeNextTx` (`PauseGuard.sol:119`), so the proposal waits until the PauseGuard's admin unpauses it. If `txExpiration` passes first, the proposal expires and has to be proposed again. **On-chain — The setup formally verified above is deployed, and the Delay Modifier's expiration period is 24 hours:** • ⏳ pending deployment. `target()` on the Delay Modifier must return the OwnerlessSafe. • ⏳ pending deployment. `guard()` on the Delay Modifier must return the PauseGuard. • ⏳ pending deployment. The guard slot (`0x4a204f620c8c5ccdca3fd54d003badd85ba500436a431f0cbda4f558c93c34c8`, `GuardManager.sol:66` in Safe v1.5.0) on the OwnerlessSafe and on the ProposerSafe must hold the ConfigLockGuard. • ⏳ pending deployment. The OwnerlessSafe's module guard slot (`0xb104e0b93118902c651344349b610029d694cfdec91c589c91ebafbcd0289947`, `ModuleManager.sol:65` in Safe v1.5.0) must hold `address(0)`. The Delay Modifier's call to the OwnerlessSafe is a module transaction, which a module guard would check (`ModuleManager.sol:94–109`). DE150-1 and LG150-3 run with it unset. • ⏳ pending deployment. `paused()` on the PauseGuard must return false. • ⏳ pending deployment. `txExpiration()` on the Delay Modifier must be 86,400 seconds (24 hours). • Assumes the OwnerlessSafe and ProposerSafe run Safe v1.5.0, the version DE150-1, DE150-2, LG150-3 and CL150 check (`GuardManager.sol:66`). **FV — The Term multisig holders on the OwnerlessSafe may execute DEVOPS_ROLE methods directly from the OwnerlessSafe:** • DE150-2 `ownerlessOwnersExecuteDevopsMethodThroughExecTransaction` ✅ passes. In one execution: ◦ An owner of the OwnerlessSafe (a Term multisig holder, P1.4) approves with `approveHash` an OwnerlessSafe transaction that calls a DEVOPS_ROLE method on a protocol contract. ◦ Anyone submits it through `execTransaction` with that approval as the signature. ◦ The call passes `onlyRole(DEVOPS_ROLE)` because the OwnerlessSafe holds `DEVOPS_ROLE` (P1.1), and the method runs. • DE150-2 sets the OwnerlessSafe's threshold to 1. By SE150-6 (P1.2), with the real threshold of 9 (P1.4), all nine owners' approvals take the same path. • DE150-2 assumes the OwnerlessSafe has no guard. Its ConfigLockGuard lets the call through. Its `lockedDelay` is the Delay Modifier (P5.DelayModifier.13), and a DEVOPS_ROLE method call to a protocol contract is a plain call to neither the OwnerlessSafe nor the Delay Modifier, so it is not a locked call. CL150-10 `checkTransactionAcceptsEverythingElse` ✅

#	Statement	Evidence
		passes: whatever `lockedDelay` is, the Delay Modifier included, the guard accepts every plain call that is not a locked call. It checks ConfigLockGuard's `checkTransaction` directly, so the Safe version does not matter. Its `checkAfterExecution` does nothing (`ConfigLockGuard.sol:141`), so DE150-2's path is unchanged.

Proposition 2 — Only TERM token holders, through a proposal made by a holder with at least 1,000 TERM of voting power that reaches a quorum of 1,000,000 TERM, if proposed within 2 hours of the transaction being queued, and Term multisig holders may veto Delay Modifier transactions within the 24-hour veto window, in which a Delay Modifier transaction cannot be executed until txCooldown has passed since it was queued, holding: 1. the owners, fallbackHandler, modules, guard, module guard constant for the DelayOwnerSafe, and 2. the owner, target, modules, guard constant for the Delay Modifier and Roles Modifier, and 3. the role targets, role scopes, role assignments constant for the Roles Modifier, and 4. the voting token, proposal threshold, voting threshold, quorum constant for the Governor, and 5. the txCooldown constant for the Delay Modifier.

#	Statement	Evidence
P2.1	Only Modules of Delay Modifier and Owner of Delay Modifier can interact with Delay Modifier, except `executeNextTx` and `skipExpired`, which anyone can call, but they can only execute or skip transactions queued by Modules of Delay Modifier or Owner of Delay Modifier.	Same as P1.6.
P2.2	Veto Delay Modifier transactions (`Delay.setTxNonce(uint256)`) is only callable by Owner of Delay Modifier, and not by Modules of Delay Modifier.	**FV — `setTxNonce`, which vetoes Delay Modifier transactions, is only callable by Owner of Delay Modifier, and not by Modules of Delay Modifier:** • DM-8 `atMostOneCallerPassesOnlyOwner` ✅ passes: `setTxNonce` is one of the Delay Modifier's ten `onlyOwner` functions, and each succeeds only when the caller is the owner (Owner of Delay Modifier). • DM-15 `enabledModuleThatIsNotOwnerCannotSetTxNonce` ✅ passes: in particular, an enabled module (Modules of Delay Modifier) that is not the owner cannot call `setTxNonce`; the call always reverts, whatever the nonce. By P1.7, the owner is not a module, so this covers every module. • DM-16 `ownerCanSetTxNonce` ✅ passes: the owner's `setTxNonce` succeeds and sets `txNonce`. So DM-8 and DM-15 do not hold just because `setTxNonce` always reverts.
P2.3	`Owner of Delay Modifier = DelayOwnerSafe`, holding: 1. the owners, fallbackHandler, modules, guard, module guard constant for the DelayOwnerSafe, and 2. the owner, target, modules, guard constant for the Delay Modifier and Roles Modifier, and 3. the role targets, role scopes, role assignments constant for the Roles Modifier, and 4. the voting token, proposal threshold, voting threshold, quorum constant for the Governor.	**FV — The Owner of Delay Modifier is the single address `owner()` returns:** • DM-8 `atMostOneCallerPassesOnlyOwner` ✅ passes: each of the Delay Modifier's ten `onlyOwner` functions succeeds only when the caller is the address `owner()` returns, and from the same state no two different callers can both succeed. So only one address at a time can act as the Owner of Delay Modifier. • DM-19 `ownerCanCallEachDelaySetting` ✅ passes: for each of the ten, some call from that address succeeds. So DM-8 is not vacuous. **On-chain — `owner()` on the Delay Modifier returns the DelayOwnerSafe:** • ⏳ pending deployment. `owner()` on the Delay Modifier must return the DelayOwnerSafe.

#	Statement	Evidence
P2.4	Only Modules of DelayOwnerSafe and Owners of DelayOwnerSafe can interact with the DelayOwnerSafe, except `execTransaction`, which anyone can call only if Owners of DelayOwnerSafe have approved the transaction, and except `fallback` and `receive`, which anyone can call but which do not change state, assuming the fallbackHandler is `address(0)`.	**FV — Callers can only interact with the Safe through `execTransactionFromModule`, `execTransactionFromModuleReturnData`, `execTransaction`, `approveHash`, `enableModule`, `disableModule`, `addOwnerWithThreshold`, `removeOwner`, `swapOwner`, `changeThreshold`, `setGuard`, `setFallbackHandler`, `setModuleGuard`, `setup`, `simulateAndRevert`, `fallback` and `receive`:** • SE150-1 `safeEntryPointsAreAllAccountedFor` ✅ passes: Safe v1.5.0 has no write functions besides `execTransactionFromModule`, `execTransactionFromModuleReturnData`, `execTransaction`, `approveHash`, `enableModule`, `disableModule`, `addOwnerWithThreshold`, `removeOwner`, `swapOwner`, `changeThreshold`, `setGuard`, `setFallbackHandler`, `setModuleGuard`, `setup`, `simulateAndRevert`, `fallback` and `receive` (the Prover checks `fallback` and `receive` as one entry), and any unsupported call lands in `fallback`. These are P1.2's seventeen entry points. Each blob below covers some of these entry points, and together they cover all seventeen. **FV — `execTransactionFromModule` and `execTransactionFromModuleReturnData` are only callable by Modules of Safe:** • SE150-3 `onlyEnabledModulesCanCallModuleExec` ✅ passes: only an enabled module (Modules of Safe) can successfully call `execTransactionFromModule` or `execTransactionFromModuleReturnData`. • SE150-4 `enabledModuleCanMakeTheSafeAct` ✅ passes: an enabled module can call them directly, with no owner signatures, and the Safe makes an outgoing call or delegatecall. So SE150-3 is not vacuous. **FV — `approveHash` is only callable by Owners of Safe:** • SE150-8 `onlyOwnersCanApproveHashes` ✅ passes: in any call, an approval is only recorded when the caller is an owner (Owners of Safe) recording its own approval. So a call to `approveHash` from anyone else records nothing. An owner's recorded approval then counts as that owner's signature in `execTransaction`. **FV — `execTransaction` is callable by anyone, but only executes once a threshold of Owners of Safe have approved:** • SE150-5 `execTransactionRunsOnlyAfterTheSignatureCheck` ✅ passes: whoever the caller is, `execTransaction` only runs once the signature check has passed for that exact transaction, asking for as many signatures as the threshold. The rule places no restriction on the caller. • SE150-12 `eachSignatureAcceptsANewApprovingOwner` ✅ passes: each signature the check accepts comes from a different owner (Owners of Safe) who approved the transaction. For an approval made through `approveHash`, SE150-8 ensures the owner recorded it itself. So with SE150-5, the Safe acts through `execTransaction` only once as many different owners as the threshold have approved. • SE150-6 `ownersCanMakeTheSafeAct` ✅ passes: once the signature threshold is passed, `execTransaction` succeeds and the Safe acts, for any threshold and any number of owners (Owners of Safe). So SE150-5 is not vacuous. **FV — `enableModule`, `disableModule`, `addOwnerWithThreshold`, `removeOwner`, `swapOwner`, `changeThreshold`, `setGuard`, `setFallbackHandler` and `setModuleGuard` are only callable from Modules of Safe or Owners of Safe:** • SE150-7 `settingsOnlyChangeWhenTheSafeCallsItself` ✅ passes: in any call, the owners, threshold, modules, guard, module guard and fallbackHandler only change when the caller is the Safe itself. So a call to any of these nine functions from anyone else changes nothing. • SC150-4 `settingsOnlyChangeThroughAModuleOrOwners` ✅ passes: over every write function, `fallback` included while no fallbackHandler is set (P2.5), a setting only changes through an enabled module's (Modules of Safe) `execTransactionFromModule` or `execTransactionFromModuleReturnData`, or through `execTransaction`. By the `execTransactionFromModule` and `execTransaction` blobs above, those paths are open only to Modules of Safe and to a threshold of Owners of Safe. • SC150-1 `moduleCanChangeSettings`, SC150-2 `moduleCanChangeSettingsWithReturnData` ✅ pass: for each function above, a module's (Modules of Safe) `execTransactionFromModule`, and likewise its

#	Statement	Evidence
		`execTransactionFromModuleReturnData`, addressed to the Safe with a call to that function, succeeds and changes a setting. • SC150-3 `ownersCanChangeSettings` ✅ passes: for each function above, an owner's (Owners of Safe) `execTransaction`, addressed to the Safe with a call to that function, succeeds and changes a setting once the threshold is passed. Together with SC150-1 and SC150-2, this shows the restriction is not achieved by nothing working. **FV — `setup` always reverts after deploy:** • SE150-9 `setupAlwaysRevertsAfterSetup` ✅ passes: once the Safe is set up, `setup` always reverts, so nobody can re-run it to replace the owners, threshold, modules or handler. • The DelayOwnerSafe is set up: its threshold is 5 and it has eleven owners (P2.6). **FV — `simulateAndRevert` always reverts:** • SE150-11 `simulateAndRevertAlwaysReverts` ✅ passes: anyone can call `simulateAndRevert`, and it does make a delegatecall, but it always reverts afterwards, so none of its effects persist. **FV — `fallback` is callable by anyone but does not change state, assuming the fallbackHandler is `address(0)`:** • SE150-21 `noHandlerFallbackChangesNoState` ✅ passes: with no handler set, `fallback` changes no state at all. It writes no storage, makes no call of any kind (call, delegatecall, staticcall or callcode) and creates or destroys no contract, and a `fallback` call that succeeds carries no ETH, so not even the Safe's balance changes. • SE150-23 `storageHookFires`, SE150-24 `callHookFires` ✅ pass: a successful `approveHash` is seen writing the Safe's storage, and `fallback` with a handler set is seen making its call. So SE150-21's checks do detect storage writes and calls. • SE150-22 `noHandlerFallbackSucceeds` ✅ passes: with no handler set, some call to `fallback` succeeds, so SE150-21 is not vacuous. **FV — `receive` is callable by anyone but does not change state:** • SE150-25 `receiveChangesNoState` ✅ passes: whatever the handler, a call that runs `receive` changes no state beyond the ETH it is sent. It writes no storage, makes no call of any kind and creates or destroys no contract. • Here, state means the Safe's storage, which holds its owners, threshold, modules, guard, module guard, fallbackHandler, nonce and approvals. The ETH sent is added to the Safe's balance, which is not part of that storage. • SE150-23 `storageHookFires`, SE150-24 `callHookFires` ✅ pass: a successful `approveHash` is seen writing the Safe's storage, and `fallback` with a handler set is seen making its call. So SE150-25's checks do detect storage writes and calls. • SE150-26 `receiveRuns` ✅ passes: some call to the Safe runs `receive` and succeeds, so SE150-25 is not vacuous. **On-chain — The DelayOwnerSafe runs Safe v1.5.0, the code the SE150 and SC150 rules check:** • ⏳ pending deployment. The DelayOwnerSafe must be a Safe v1.5.0 proxy: `VERSION()` must return `1.5.0`, and storage slot 0 (the proxy's `singleton`) must hold the canonical Safe v1.5.0 singleton. `VERSION()` alone does not tell Safe v1.5.0 from SafeL2 v1.5.0; the singleton does. • So the evidence above, P1.2's, holds for the DelayOwnerSafe, which runs the same Safe v1.5.0 code as the OwnerlessSafe and ProposerSafe (P1.9). • Assumes the DelayOwnerSafe runs Safe v1.5.0, the version SE150 and SC150 check.
P2.5	DelayOwnerSafe fallbackHandler is `address(0)`, holding: 1. the owners, fallbackHandler, modules, guard, module guard constant for the DelayOwnerSafe, and 2. the owner, target, modules, guard constant for the Delay Modifier and Roles	**On-chain — DelayOwnerSafe fallbackHandler is `address(0)`:** • ⏳ pending deployment. The fallbackHandler slot (`0x6c9a6c4a39284e37ed1cf53d337577d14212a4870fb976a4366c693b939918d5`, `FallbackManager.sol:13` in Safe v1.5.0) on the DelayOwnerSafe must hold `address(0)`. • Assumes the DelayOwnerSafe runs Safe v1.5.0 (`FallbackManager.sol:13`).

#	Statement	Evidence
	Modifier, and 3. the role targets, role scopes, role assignments constant for the Roles Modifier, and 4. the voting token, proposal threshold, voting threshold, quorum constant for the Governor.	
P2.6	`Owners of DelayOwnerSafe = Term multisig holders on DelayOwnerSafe`, holding: 1. the owners, fallbackHandler, modules, guard, module guard constant for the DelayOwnerSafe, and 2. the owner, target, modules, guard constant for the Delay Modifier and Roles Modifier, and 3. the role targets, role scopes, role assignments constant for the Roles Modifier, and 4. the voting token, proposal threshold, voting threshold, quorum constant for the Governor.	**On-chain — Owners of DelayOwnerSafe are eleven Term multisig holders, and five of them must approve each transaction:** • ⏳ pending deployment. The DelayOwnerSafe must run Safe v1.5.0 (`VERSION()`), with threshold 5 (`getThreshold`) and eleven owners (`getOwners`), each an externally owned account, with no contract code, held by a Term multisig holder. • Assumes the DelayOwnerSafe runs Safe v1.5.0. • The identities of the eleven key holders are not visible on chain. Their status as Term multisig holders is a deployment fact.
P2.7	`Modules of DelayOwnerSafe = Roles Modifier`, holding: 1. the owners, fallbackHandler, modules, guard, module guard constant for the DelayOwnerSafe, and 2. the owner, target, modules, guard constant for the Delay Modifier and Roles Modifier, and 3. the role targets, role scopes, role assignments constant for the Roles Modifier, and 4. the voting token, proposal threshold, voting threshold, quorum constant for the Governor.	**On-chain — The Modules of DelayOwnerSafe list contains only the Roles Modifier:** • ⏳ pending deployment. `getModulesPaginated` on the DelayOwnerSafe must return just the Roles Modifier, and report that the list ends there. • Assumes the DelayOwnerSafe runs Safe v1.5.0.
P2.8	Only Roles Modifier Modules with AssignedRole or Roles Modifier Owner can interact with Roles Modifier.	**FV — Callers can only interact with the Roles Modifier through `execTransactionFromModule`, `execTransactionFromModuleReturnData`, `execTransactionWithRole`, `execTransactionWithRoleReturnData`, `setMultisend`, `allowTarget`, `revokeTarget`, `scopeTarget`, `scopeAllowFunction`, `scopeRevokeFunction`, `scopeFunction`, `scopeFunctionExecutionOptions`, `scopeParameter`, `scopeParameterAsOneOf`, `unscopeParameter`, `assignRoles`, `setDefaultRole`, `setAvatar`, `setTarget`, `enableModule`, `disableModule`, `setGuard`, `transferOwnership`, `renounceOwnership` and `setUp`:** • RI-14 `rolesWriteFunctionsAreTheKnownTwentyFive` ✅ passes: the Roles Modifier has no `fallback` or `receive` and no write functions besides these twenty-five. Each blob below covers some of these, and together they cover all twenty-five. **FV — `execTransactionFromModule`, `execTransactionFromModuleReturnData`, `execTransactionWithRole` and `execTransactionWithRoleReturnData` allow only Roles Modifier Modules with AssignedRole to execute transactions through Roles Modifier:** • RI-7 `onlyEnabledModulesCanExec` ✅ passes: only enabled modules can call `execTransactionFromModule`, `execTransactionFromModuleReturnData`, `execTransactionWithRole` and `execTransactionWithRoleReturnData`. • RI-8 `moduleWithoutDefaultRoleCannotExecFromModule` ✅ passes: an enabled module that is not a member of its default role cannot execute through `execTransactionFromModule` or `execTransactionFromModuleReturnData`.

#	Statement	Evidence
		• RI-9 `moduleWithoutRoleCannotExecTransactionWithRole`, RI-10 `moduleWithoutRoleCannotExecTransactionWithRoleReturnData` ✅ pass: an enabled module that is not a member of the role the call names cannot execute through `execTransactionWithRole` or `execTransactionWithRoleReturnData`. With RI-8, a module with no assigned role can execute nothing through any of the four. • RI-11 `roleMemberModuleCanExecFromModule`, RI-12 `roleMemberModuleCanExecTransactionWithRole`, RI-13 `roleMemberModuleCanExecTransactionWithRoleReturnData` ✅ pass: an enabled module that is a member of the role the call runs under (Roles Modifier Modules with AssignedRole) can successfully call each of the four, so the restriction is not achieved by nothing working. **FV — `setMultisend`, `allowTarget`, `revokeTarget`, `scopeTarget`, `scopeAllowFunction`, `scopeRevokeFunction`, `scopeFunction`, `scopeFunctionExecutionOptions`, `scopeParameter`, `scopeParameterAsOneOf`, `unscopeParameter`, `assignRoles`, `setDefaultRole`, `setAvatar`, `setTarget`, `enableModule`, `disableModule`, `setGuard`, `transferOwnership` and `renounceOwnership` are only callable by Roles Modifier Owner:** • RI-5 `onlyOwnerCanCallRolesSettings` ✅ passes: each of these twenty functions succeeds only when the caller is the owner (Roles Modifier Owner). • RI-2 `rolesConfigOnlyChangesThroughOwner` ✅ passes: over every write function, the four execution functions included, the owner, guard, multisend, avatar, target, module list, default roles, role membership and target clearances only change when the caller is the owner. So no other entry point offers a way around RI-5. • RI-6 `ownerCanCallEachRolesSetting` ✅ passes: for each of the twenty, some call from the owner succeeds. So RI-5 is not vacuous. **FV — `setUp` always reverts after deploy:** • RI-1 `setUpAlwaysRevertsAfterDeployment` ✅ passes: once the module list is set up, `setUp` always reverts, so nobody can re-run it to replace the owner or the module list. • The Roles Modifier's module list is set up: it holds the Governor (P2.10).
P2.9	Only Roles Modifier Modules with AssignedRole can execute transactions through Roles Modifier. Roles Modifier Owner cannot do so.	**FV — `execTransactionFromModule`, `execTransactionFromModuleReturnData`, `execTransactionWithRole` and `execTransactionWithRoleReturnData` allow only Roles Modifier Modules with AssignedRole to execute transactions through Roles Modifier:** • RI-7 `onlyEnabledModulesCanExec` ✅ passes: only enabled modules can call `execTransactionFromModule`, `execTransactionFromModuleReturnData`, `execTransactionWithRole` and `execTransactionWithRoleReturnData`. • RI-8 `moduleWithoutDefaultRoleCannotExecFromModule` ✅ passes: an enabled module that is not a member of its default role cannot execute through `execTransactionFromModule` or `execTransactionFromModuleReturnData`. • RI-9 `moduleWithoutRoleCannotExecTransactionWithRole`, RI-10 `moduleWithoutRoleCannotExecTransactionWithRoleReturnData` ✅ pass: an enabled module that is not a member of the role the call names cannot execute through `execTransactionWithRole` or `execTransactionWithRoleReturnData`. With RI-8, a module with no assigned role can execute nothing through any of the four. • RI-11 `roleMemberModuleCanExecFromModule`, RI-12 `roleMemberModuleCanExecTransactionWithRole`, RI-13 `roleMemberModuleCanExecTransactionWithRoleReturnData` ✅ pass: an enabled module that is a member of the role the call runs under (Roles Modifier Modules with AssignedRole) can successfully call each of the four, so the restriction is not achieved by nothing working. **FV — Roles Modifier Owner cannot execute transactions through Roles Modifier unless it is one of the Roles Modifier Modules with AssignedRole:** • RI-22 `ownerWithoutModuleRoleCannotExecFromModule`, RI-23 `ownerWithoutModuleRoleCannotExecTransactionWithRole`, RI-24 `ownerWithoutModuleRoleCannotExecTransactionWithRoleReturnData` ✅ pass: on all four execution entry points, the

#	Statement	Evidence
		owner's (Roles Modifier Owner) call reverts unless the owner is itself an enabled module and a member of the role the call runs under (its default role for the two FromModule functions, the role it names for the two WithRole functions). **On-chain — Roles Modifier Owner is not also a Roles Modifier Module:** • ⏳ pending deployment. `owner()` on the Roles Modifier must return `address(0)`, and its `_owner` slot (slot 51, `OwnableUpgradeable`) must hold zero: whoever deploys and configures the Roles Modifier must renounce ownership once Role 1 is set up. • `address(0)` can never be an enabled module: `enableModule` rejects it (zodiac `Modifier.sol:87`). • ⏳ pending deployment. `isModuleEnabled(address(0))` on the Roles Modifier must return false, and its module list must hold only the Governor. • So the owner is not a module, and by RI-22 to RI-24 it cannot execute transactions through the Roles Modifier.
P2.10	`Roles Modifier Modules with AssignedRole = Governor`, holding: 1. the owners, fallbackHandler, modules, guard, module guard constant for the DelayOwnerSafe, and 2. the owner, target, modules, guard constant for the Delay Modifier and Roles Modifier, and 3. the role targets, role scopes, role assignments constant for the Roles Modifier, and 4. the voting token, proposal threshold, voting threshold, quorum constant for the Governor.	**On-chain — The Roles Modifier Modules list contains only the Governor:** • ⏳ pending deployment. `getModulesPaginated` on the Roles Modifier must return just the Governor, and report that the list ends there. • ⏳ pending deployment. The Roles Modifier's complete event history, from its creation, must agree: one `EnabledModule`, for the Governor, and no `DisabledModule`. **On-chain — The Governor is a Roles Modifier Module with an AssignedRole:** • ⏳ pending deployment. The Governor must be recorded as a member of Role 1 (`roles[1].members`, storage slot 107). • So the Governor is the only Roles Modifier Module, and it has an assigned role: Roles Modifier Modules with AssignedRole = Governor.
P2.11	Governor is assigned only Role 1, holding: 1. the owners, fallbackHandler, modules, guard, module guard constant for the DelayOwnerSafe, and 2. the owner, target, modules, guard constant for the Delay Modifier and Roles Modifier, and 3. the role targets, role scopes, role assignments constant for the Roles Modifier, and 4. the voting token, proposal threshold, voting threshold, quorum constant for the Governor.	**FV — A Roles Modifier Module's roles change only through `assignRoles`, and its default role only through `setDefaultRole`, and each change emits an event:** • RI-19 `membershipOnlyChangesThroughAssignRoles` ✅ passes: over every write function, `setUp` included, and any caller, a Roles Modifier Module's membership of a role changes only through `assignRoles` (`Roles.sol:290`), and every change emits `AssignRoles`. • RI-20 `defaultRoleOnlyChangesThroughSetDefaultRole` ✅ passes: likewise, a module's default role changes only through `setDefaultRole` (`Roles.sol:310`), and every change emits `SetDefaultRole`. **On-chain — Governor is assigned only Role 1:** • ⏳ pending deployment. The Roles Modifier's complete event history, from its creation, must have exactly one `AssignRoles`: the Governor, Role 1, member. By RI-19, that is every membership change since creation, so the Governor is a member of Role 1 and of no other role. • ⏳ pending deployment. The same history must have exactly one `SetDefaultRole`: the Governor, Role 1. By RI-20, the Governor's default role is Role 1, so its `execTransactionFromModule` calls also run under Role 1.
P2.12	Only Holders of Governor Vote Token can interact with the Governor, except `execute`, which anyone can call only if Holders of Governor Vote Token proposed and passed the proposal, and except the `castVote` functions (`castVote`, `castVoteWithReason`, `castVoteWithReasonAndParams`, `castVoteBySig`,	**FV — Callers can only interact with the Governor through `propose`, `queue`, `execute`, `cancel`, `castVote`, `castVoteWithReason`, `castVoteWithReasonAndParams`, `castVoteBySig`, `castVoteWithReasonAndParamsBySig`, `relay`, `onERC721Received`, `onERC1155Received`, `onERC1155BatchReceived` and `receive`:** • GS-1 `governorWriteFunctionsAreTheKnownFourteen` ✅ passes: the Governor has no write functions besides `propose`, `queue`, `execute`, `cancel`, `castVote`, `castVoteWithReason`, `castVoteWithReasonAndParams`, `castVoteBySig`, `castVoteWithReasonAndParamsBySig`, `relay`, `onERC721Received`, `onERC1155Received`, `onERC1155BatchReceived` and `receive`. Each blob below covers some of these, and together they cover all fourteen.

#	Statement	Evidence
	`castVoteWithReasonAndParamsBySig`), which anyone can call but which count a non-holder's vote as 0, and except `receive`, `onERC721Received`, `onERC1155Received` and `onERC1155BatchReceived`, which anyone can call but which make no storage writes.	**FV — `propose` is only callable by Holders of Governor Vote Token with at least the proposal threshold of voting power:** • Holders of Governor Vote Token are the source of all voting power in the Governor, including the power a delegate votes with. The Governor counts TERM's `getPastVotes` (GV-4, P2.13). TERM is an `ERC20VotesUpgradeable` token (`TermToken.sol:15`), so a holder votes with its own TERM by delegating to itself, or delegates it to another address. A delegate then votes with the holders' TERM: its power comes only from the holders who delegated to it, and they can move it elsewhere at any time. So every vote, and every proposal over the threshold, rests on TERM that holders chose to delegate. • GP-1 `callerBelowProposalThresholdCannotPropose` ✅ passes: any caller whose voting power at `clock() - 1` is below the proposal threshold cannot create a proposal. • GS-3 `governorSettingsAreTheDeployedValues` ✅ passes: the proposal threshold is fixed at 1,000 tokens in the Governor's code. It is above zero, so by GP-1 no non-holder can propose. • GV-1 `holderAboveThresholdCanProposeVeto` ✅ passes: a caller with at least the proposal threshold of voting power can propose the veto. So GP-1 is not vacuous. **FV — `cancel` is only callable by the proposer, who is a Holder of Governor Vote Token:** • GS-5 `onlyTheProposerCanCancel` ✅ passes: a successful `cancel` came from the address recorded as the proposal's proposer. By GP-1, only Holders of Governor Vote Token can have proposed. • GS-6 `proposerCanCancel` ✅ passes: the proposer can cancel, so GS-5 is not vacuous. **FV — The `castVote` functions are callable by anyone, but a non-holder's vote counts as 0:** • GC-1 `directVoteCountsOnlyTheVotersVotes`, GC-2 `voteBySigCountsOnlyTheVotersVotes`, GC-3 `voteWithReasonAndParamsBySigCountsOnlyTheVotersVotes` ✅ pass: through each of `castVote`, `castVoteWithReason`, `castVoteWithReasonAndParams`, `castVoteBySig` and `castVoteWithReasonAndParamsBySig`, a proposal's tally grows by exactly the voter's votes at its snapshot, so a caller with none moves nothing. • GC-4 `nothingButAVoteMovesATally` ✅ passes: no other write function moves a tally. • GC-5 `termHolderVoteCounts` ✅ passes: a caller with votes at the snapshot can vote, and its vote moves the tally. So GC-1 to GC-3 are not vacuous. • As in the `propose` blob, all voting power comes from holders' TERM. A caller that holds no TERM and has none delegated to it has no votes, so its vote counts as 0. **FV — `queue` never succeeds:** • GS-4 `queueAlwaysReverts` ✅ passes: every call to `queue` reverts, so it never sets a proposal's eta. **FV — `execute` is callable by anyone, but only executes proposals Holders of Governor Vote Token proposed and passed:** • GS-10 `executeOnlyRunsSucceededProposals` ✅ passes: a successful `execute` ran a proposal whose state was `Succeeded`. • GS-9 `noProposalIsQueued` ✅ passes: no proposal is ever `Queued`, the other state `execute` accepts, because the only write to a proposal's eta is in `queue`, which never succeeds (GS-4). • So `execute` only runs a proposal created by a Holder of Governor Vote Token with at least the proposal threshold of voting power (GP-1), and passed on Governor Vote Token votes (GC-1 to GC-4). • GV-2 `passedVetoReachesTheDelayOwnerSafe` ✅ passes: a proposal in state `Succeeded` executes. So GS-10 is not vacuous. **FV — `relay` can only run as an action of an executed proposal, so only a passed proposal by Holders of Governor Vote Token can reach it:**

#	Statement	Evidence
		• GS-11 `relayOnlyCallableByTheGovernor` ✅ passes: a successful `relay` came from the Governor itself. • GS-15 `governorOnlyCallsItselfFromExecuteAndRelay` ✅ passes: no write function except `execute` and `relay` makes a call to the Governor, so the Governor only calls itself as an action of an executed proposal. • By the `execute` blob above, an executed proposal was proposed and passed by Holders of Governor Vote Token. • GS-12 `relayCanRun` ✅ passes: the Governor can relay, so GS-11 is not vacuous. **FV — `receive` is callable by anyone but makes no storage writes:** • GS-7 `receiveChangesNoState` ✅ passes: a successful `receive` writes no storage, makes no call of any kind and creates or destroys no contract. • GS-8 `receiveRuns` ✅ passes: some call to `receive` succeeds, so GS-7 is not vacuous. **FV — `onERC721Received`, `onERC1155Received` and `onERC1155BatchReceived` are callable by anyone but make no storage writes:** • GS-13 `tokenReceiptChangesNoState` ✅ passes: a successful call to any of the three writes no storage, makes no call of any kind and creates or destroys no contract. • GS-14 `tokenReceiptRuns` ✅ passes: each can succeed, so GS-13 is not vacuous.
P2.13	`Governor Vote Token = TERM token`, holding: 1. the owners, fallbackHandler, modules, guard, module guard constant for the DelayOwnerSafe, and 2. the owner, target, modules, guard constant for the Delay Modifier and Roles Modifier, and 3. the role targets, role scopes, role assignments constant for the Roles Modifier, and 4. the voting token, proposal threshold, voting threshold, quorum constant for the Governor.	**On-chain — The Governor's vote token is the TERM token:** • ⏳ pending deployment. `token()` on the Governor must return the TERM token (`0xC3d21f79C3120A4fFda7A535f8005a7c297799bF`). **FV — The Governor counts votes in that token:** • GV-4 `governorVotesAreTermPastVotes` ✅ passes: for any account and any past timepoint, the Governor's vote count is its vote token's `getPastVotes`. So the Governor Vote Token is the TERM token.
P2.14	The TERM token holders, through a proposal made by a holder with at least 1,000 TERM of voting power that reaches a quorum of 1,000,000 TERM, if proposed within 2 hours of the transaction being queued, and the Term multisig holders on the DelayOwnerSafe may veto Delay Modifier transactions within the 24-hour veto window, in which a Delay Modifier transaction cannot be executed until txCooldown has passed since it was queued.	**FV — A Delay Modifier transaction cannot be executed until txCooldown has passed since it was queued:** • DP-31 `onlyExecuteNextTxReachesTheTarget` ✅ passes: over every write function of the Delay Modifier and the PauseGuard, only `executeNextTx` hands a transaction to the Delay Modifier's target. So `executeNextTx` is the only way a queued transaction is executed. • DP-25 `executeNextTxConsumesOnlyTheEntryAtTxNonce` ✅ passes: `executeNextTx` only executes the queue entry at `txNonce`, and moves `txNonce` forward by one. • DP-27 `executeNextTxRevertsDuringCooldown` ✅ passes: until `txCooldown` has passed since the entry at `txNonce` was queued, `executeNextTx` always reverts, whoever calls it and whatever it is given. With DP-25, no queue entry can be executed before its own cooldown has passed. This is the veto window. • DP-26 `queuedTransactionExecutesAfterCooldown` ✅ passes: once the cooldown has passed, before the entry expires and with the PauseGuard not paused, anyone's `executeNextTx` executes it. So DP-27 is not vacuous, and the veto window ends when the cooldown does. • `executeNextTx` compares against `txCooldown` as it is when `executeNextTx` runs, not as it was when the transaction was queued (`Delay.sol:208–211`). So the veto window lasts `txCooldown` only while `txCooldown` is held constant, as Proposition 2 holds it. `txExpiration` only limits how long after the cooldown a transaction can still be executed (`Delay.sol:212–218`), so it does not affect the veto window. **FV — Once a veto moves `txNonce` past a transaction, that transaction can never be executed:**

#	Statement	Evidence
		• DP-29 `vetoedTransactionCannotExecute` ✅ passes: a module queues a transaction, the owner's `setTxNonce` moves `txNonce` past it, and `executeNextTx` with that transaction then reverts. • DP-30 `entryPassedByTxNonceNeverExecutes` ✅ passes: from any state, once `txNonce` is past a queue entry, `executeNextTx` with that entry's transaction reverts, unless the same transaction has been queued again and sits at `txNonce`. Queuing it again makes a new entry, with its own veto window. • DP-24 `txNonceNeverDecreases` ✅ passes: no write function moves `txNonce` backwards, so a vetoed entry stays vetoed, whatever later happens to `txCooldown` or `txExpiration`. **On-chain — The veto window is 24 hours:** • ⏳ pending deployment. `txCooldown()` on the Delay Modifier must be 86,400 seconds (24 hours). **FV — The TERM token holders can veto Delay Modifier transactions within the 24-hour veto window, through a proposal made by a holder with at least the proposal threshold of voting power that reaches the quorum, proposed to the Governor within 2 hours of the transaction being queued and executed once it passes the vote:** • GE-1 `termHoldersVetoThroughTheGovernor` ✅ passes. In one execution: ◦ A TERM token holder proposes the veto to the Governor: `setTxNonce(n)` on the Delay Modifier, for an `n` the Delay Modifier accepts. ◦ A TERM token holder votes for it. The proposal starts with no votes, so the Governor's own counting of TERM votes is what passes it. ◦ Once the vote has ended, anyone's `execute` succeeds with the proposal `Succeeded`. The call goes through the Roles Modifier and the DelayOwnerSafe to the Delay Modifier, whose owner is the DelayOwnerSafe (P2.3). ◦ The Delay Modifier's `txNonce` becomes `n`. • GE-1's proposal meets both bars. Its `propose` succeeds, so by GP-1 (P2.12) the proposer has at least the proposal threshold of voting power. It is `Succeeded`, so by GS-17 its For and Abstain votes reach the quorum. • GS-17 `succeededProposalReachedQuorum` ✅ passes: a proposal is `Succeeded` only once its For and Abstain votes together reach the quorum at its snapshot. • GS-18 `proposalCanSucceedWithAPositiveQuorum` ✅ passes: a proposal can be `Succeeded` with a quorum above zero to reach. So GS-17 is not vacuous. • In GE-1, the DelayOwnerSafe is a stand-in that forwards the Roles Modifier's call. GV-3 `delayOwnerSafeLandsTheVeto` ✅ passes: on the real Safe v1.5.0 with no module guard set, an enabled module's `execTransactionFromModule(Delay Modifier, 0, setTxNonce(n), Call)` succeeds and sets the Delay Modifier's `txNonce` to `n`. The Roles Modifier is the DelayOwnerSafe's module (P2.7), so the real DelayOwnerSafe lands the veto the same way. • The veto lands within the 24-hour veto window. The Governor's voting delay is 0 and its voting period is 79,200 seconds (22 hours), on a timestamp clock. GS-2 `governorSettingsNeverChange` and GS-3 `governorSettingsAreTheDeployedValues` ✅ pass: no function changes the voting delay or voting period, and the Governor's code fixes them at those values. A proposal made at time `p` can be executed from `p + 79,201`, once its deadline has passed, and the veto window closes 86,400 seconds after the transaction is queued. So a veto proposed within 7,198 seconds (just under 2 hours) of the transaction being queued can pass and execute within the veto window. • DP-32 `ownerCanSetTxNonceAtAnyTime` ✅ passes: whatever the time, `txCooldown`, `txExpiration` and pause state, the Delay Modifier owner's `setTxNonce(n)` succeeds and sets `txNonce` to `n`, for any `n` the Delay Modifier accepts. So the veto lands whenever the Governor's `execute` runs. • GE-1 assumes the Roles Modifier's target is the DelayOwnerSafe and its guard is the SetTxNonceGuard (read below). **On-chain — The setup formally verified above is deployed:** • ⏳ pending deployment. `target()` on the Roles Modifier must return the DelayOwnerSafe, and `guard()` must return the SetTxNonceGuard. • ⏳ pending deployment. The DelayOwnerSafe's module guard slot

#	Statement	Evidence
		(`0xb104e0b93118902c651344349b610029d694cfdec91c589c91ebafbcd0289947`, `ModuleManager.sol:65` in Safe v1.5.0) must hold `address(0)`. The Roles Modifier's call to the DelayOwnerSafe, which lands the TERM token holders' veto, is a module transaction, which a module guard would check (`ModuleManager.sol:94–109`). GV-3 runs with it unset. • ⏳ pending deployment. The Governor must be deployed from term-finance-ops-contracts' `TermFinanceProtocolGovernor.sol` at commit `5d90281be203389bdc465963164f635048e5f3e6` (PR 1096), the code GS-3 checks: its voting delay of 0, voting period of 79,200 seconds and timestamp clock are fixed in that code. **FV — The proposal threshold is 1,000 TERM of voting power and the quorum is 1,000,000 TERM:** • GS-3 `governorSettingsAreTheDeployedValues` ✅ passes: `proposalThreshold()` is 1,000 TERM (1e21), and `quorum(t)` is 1,000,000 TERM (1e24) at every timepoint `t`, both fixed in the Governor's code. **FV — The Term multisig holders on the DelayOwnerSafe can veto Delay Modifier transactions within the 24-hour veto window, directly from the DelayOwnerSafe:** • DV150-1 `approvedOwnersVetoThroughExecTransaction` ✅ passes. In one execution: ◦ An owner of the DelayOwnerSafe (a Term multisig holder, P2.6) approves a `setTxNonce(n)` transaction with `approveHash`, for an `n` the Delay Modifier accepts. ◦ `execTransaction` runs it with that approval as the signature. ◦ With the DelayOwnerSafe as the Delay Modifier's owner (P2.3), the call succeeds and sets the Delay Modifier's `txNonce` to `n`. • DV150-1 sets the DelayOwnerSafe's threshold to 1. By SE150-6 (P2.4), with the real threshold of 5 (P2.6), five owners' approvals take the same path. • DV150-1 assumes the DelayOwnerSafe owns the Delay Modifier (P2.3) and has no guard. The DelayOwnerSafe's guard is ConfigLockGuard (read below), which the next two bullets cover. The holders can veto at any point in the 24-hour veto window: by DP-32 (above), the owner's `setTxNonce` succeeds whatever the time. • ConfigLockGuard lets the veto through. CL150-10 `checkTransactionAcceptsEverythingElse` ✅ passes: whatever `lockedDelay` is, the guard accepts every plain call that is not a locked call. The veto, `setTxNonce(n)` on the Delay Modifier, is a plain call. It is not one of the nine calls the guard locks on its locked modifier (CL150-6 to CL150-9, CL150-24). The guard's `checkAfterExecution` does nothing (`ConfigLockGuard.sol:141`), so DV150-1's path is unchanged. • OD150-4 `guardedSafeCanStillVeto` ✅ passes. In this rule, ConfigLockGuard is installed on the DelayOwnerSafe with the Delay Modifier as its locked modifier, and the DelayOwnerSafe owns the Delay Modifier. An `execTransaction` to the Delay Modifier of `setTxNonce(n)` succeeds and `txNonce` becomes `n`, for an `n` the Delay Modifier accepts. • OD150-4 stubs the signature check. By P2.4, `execTransaction` only acts once five Owners of DelayOwnerSafe, the Term multisig holders (P2.6), have approved (SE150-5, SE150-12), and their approvals pass it (SE150-6). **On-chain — The setup formally verified above is deployed:** • ⏳ pending deployment. The DelayOwnerSafe's guard slot (`0x4a204f620c8c5ccdca3fd54d003badd85ba500436a431f0cbda4f558c93c34c8`, `GuardManager.sol:66` in Safe v1.5.0) must hold ConfigLockGuard. This is the same requirement as P5.DelayOwnerSafe.10. • ⏳ pending deployment. `lockedDelay()` on ConfigLockGuard must return the Delay Modifier. • Assumes the DelayOwnerSafe runs Safe v1.5.0, the version CL150 and OD150-4 check (`GuardManager.sol:66`).

Proposition 3 — Only the Term multisig holders on the PauseSafe may pause, and only the Term multisig holders on the ProposerSafe may unpause, the execution of Delay Modifier transactions, and if a pause outlasts the 24-hour expiration period following the cooldown, the Delay Modifier transaction can only be skipped or vetoed, holding: 1. the owners, fallbackHandler,

modules, guard constant for the PauseSafe and ProposerSafe, and 2. the guard, txCooldown, txExpiration constant for the Delay Modifier, and 3. the pauser and admin constant for the PauseGuard.

#	Statement	Evidence
P3.1	Only the PauseGuard pauser and the PauseGuard admin can interact with the PauseGuard, except `checkTransaction` and `checkAfterExecution`, which anyone can call but which do not change state.	**FV — Callers can only change the PauseGuard through `pause`, `unpause` and `setPauser`:** • DP-9 `pausedOnlyChangesThroughPauseOrUnpause` ✅ passes: over every write function of the Delay Modifier and the PauseGuard, `paused` changes only through `pause` or `unpause`. • DP-13 `pauserOnlyChangesThroughSetPauser` ✅ passes: over every write function of both contracts, the pauser only changes through `setPauser`. • DP-12 `pauseGuardWriteFunctionsAreTheKnownThree` ✅ passes: PauseGuard has no fallback, and its write functions are exactly `pause`, `unpause` and `setPauser`. • `checkTransaction` and `checkAfterExecution` are `view` functions (`PauseGuard.sol:117`, `PauseGuard.sol:123`), so they change no state. **FV — `pause` is only callable by the PauseGuard pauser, and `unpause` and `setPauser` only by the PauseGuard admin:** • DP-3 `pauseRevertsForAnyoneButThePauser` ✅ passes: anyone who is not the pauser is rejected. • DP-6 `unpauseRevertsForAnyoneButTheAdmin` ✅ passes: anyone who is not the admin is rejected. • DP-10 `setPauserRevertsForAnyoneButTheAdmin` ✅ passes: anyone who is not the admin is rejected.

Pause

P3.Pause: Only the Term multisig holders on the PauseSafe may pause the execution of Delay Modifier transactions.

#	Statement	Evidence
P3.Pause.1	Pause Delay Modifier executions (`PauseGuard.pause()`) is only callable by the PauseGuard pauser, and not by the PauseGuard admin.	**FV — `pause` is only callable by the PauseGuard pauser, and not by the PauseGuard admin:** • DP-3 `pauseRevertsForAnyoneButThePauser` ✅ passes: anyone who is not the pauser is rejected. • DP-4 `adminAloneCannotPause` ✅ passes: being the admin does not let a caller pause. • DP-5 `pauseSucceedsForThePauser` ✅ passes: from an unpaused state, the pauser's `pause()` succeeds and `paused` is then true. So DP-3 and DP-4 do not hold just because `pause` always reverts.
P3.Pause.2	`PauseGuard pauser = PauseSafe`, holding: 1. the owners, fallbackHandler, modules, guard constant for the PauseSafe and ProposerSafe, and 2. the guard, txCooldown, txExpiration constant for the Delay Modifier, and 3. the pauser and admin constant for the PauseGuard.	**On-chain — The PauseGuard's pauser is the PauseSafe:** • ⏳ pending deployment. `pauser()` on the PauseGuard must return the PauseSafe.
P3.Pause.3	Only Modules of PauseSafe and Owners of PauseSafe can interact with the PauseSafe, except `execTransaction`, which anyone can call only if Owners of PauseSafe have approved the transaction, and except `fallback` and `receive`, which anyone can call but which do not change state,	Same as P2.4. The PauseSafe runs Safe v1.5.0, like the DelayOwnerSafe (P3.Pause.5).

#	Statement	Evidence
	assuming the fallbackHandler is `address(0)`.	
P3.Pause.4	PauseSafe fallbackHandler is `address(0)`, holding: 1. the owners, fallbackHandler, modules, guard constant for the PauseSafe and ProposerSafe, and 2. the guard, txCooldown, txExpiration constant for the Delay Modifier, and 3. the pauser and admin constant for the PauseGuard.	**On-chain — PauseSafe fallbackHandler is `address(0)`:** • ⏳ pending deployment. The fallbackHandler slot (`0x6c9a6c4a39284e37ed1cf53d337577d14212a4870fb976a4366c693b939918d5`, `FallbackManager.sol:13` in Safe v1.5.0) on the PauseSafe must hold `address(0)`. • Assumes the PauseSafe runs Safe v1.5.0 (`FallbackManager.sol:13`).
P3.Pause.5	`Owners of PauseSafe = Term multisig holders on PauseSafe`, holding: 1. the owners, fallbackHandler, modules, guard constant for the PauseSafe and ProposerSafe, and 2. the guard, txCooldown, txExpiration constant for the Delay Modifier, and 3. the pauser and admin constant for the PauseGuard.	**On-chain — Owners of PauseSafe are nine Term multisig holders, and two of them must approve each transaction:** • ⏳ pending deployment. The PauseSafe must run Safe v1.5.0 (`VERSION()`), with threshold 2 (`getThreshold`) and nine owners (`getOwners`), each an externally owned account, with no contract code, held by a Term multisig holder. • Assumes the PauseSafe runs Safe v1.5.0.
P3.Pause.6	`Modules of PauseSafe = null`, holding: 1. the owners, fallbackHandler, modules, guard constant for the PauseSafe and ProposerSafe, and 2. the guard, txCooldown, txExpiration constant for the Delay Modifier, and 3. the pauser and admin constant for the PauseGuard.	**On-chain — The Modules of PauseSafe list is empty:** • ⏳ pending deployment. `getModulesPaginated` on the PauseSafe must return an empty list. • Assumes the PauseSafe runs Safe v1.5.0.
P3.Pause.7	The Term multisig holders on the PauseSafe may pause the execution of Delay Modifier transactions, and if a pause outlasts the 24-hour expiration period following the cooldown, the Delay Modifier transaction can only be skipped or vetoed.	**FV — The PauseSafe's owners can have it pause the PauseGuard, and a pause stops every Delay Modifier execution:** • DP-5 `pauseSucceedsForThePauser` ✅ passes: from an unpaused state, the pauser's `pause()` succeeds and `paused` is then true. • SE150-6 `ownersCanMakeTheSafeAct` ✅ passes: once the signature threshold is passed, `execTransaction` succeeds and the Safe acts, for any threshold and any number of owners. So the PauseSafe's owners (P3.Pause.5) can have it call `pause` on the PauseGuard. • DP-15 `pausedBlocksExecuteNextTx` ✅ passes: with the PauseGuard installed and paused, `executeNextTx` reverts and nothing reaches the Delay Modifier's target, for any transaction arguments and any queue state. • DP-16 `checkTransactionRevertsWhilePaused` ✅ passes: called directly, a paused PauseGuard rejects every transaction from every caller. • DP-31 `onlyExecuteNextTxReachesTheTarget` ✅ passes: over every write function of the Delay Modifier and the PauseGuard, only `executeNextTx` hands a transaction to the Delay Modifier's target. With DP-15, a pause stops every transaction the Delay Modifier can send. **FV — If a pause outlasts the 24-hour expiration period following the cooldown, the Delay Modifier transaction can only be skipped or vetoed:** • DP-15 `pausedBlocksExecuteNextTx` ✅ passes (above): while the PauseGuard is paused, `executeNextTx` reverts, so the transaction cannot be executed during the pause. • DP-28 `executeNextTxRevertsAfterExpiration` ✅ passes: with a non-zero `txExpiration`, once more than

#	Statement	Evidence
		txCooldown plus txExpiration has passed since the head entry was queued, executeNextTx with it reverts. That stays true at every later time, so a transaction whose window ended during the pause cannot be executed after the unpause either. • DM-12 anyoneCanSkipExpired ✅ passes: whether a call to executeNextTx or skipExpired succeeds, and what it changes, does not depend on who the caller is. DM-18 outsiderCanSkipExpired ✅ passes: a caller that is neither the owner nor a module can skip a queue entry. • DP-33 skipExpiredSucceedsWhilePaused ✅ passes: with the PauseGuard installed and paused, once the head entry has expired, anyone's skipExpired succeeds, moves txNonce past it and hands nothing to the Delay Modifier's target. So a transaction whose window ended during the pause can be skipped during the pause. • The Owner of Delay Modifier can also move txNonce past it with a veto, setTxNonce (P2.2). • DP-30 entryPassedByTxNonceNeverExecutes ✅ passes: once txNonce is past the entry, executeNextTx with that entry's transaction reverts, unless the same transaction has been queued again and sits at txNonce. Queuing it again makes a new entry, with its own veto window. **On-chain — The PauseGuard is the Delay Modifier's guard, and it is not paused:** • ⏳ pending deployment. guard() on the Delay Modifier must return the PauseGuard, as in P1.13. • ⏳ pending deployment. paused() on the PauseGuard must return false. **On-chain — The Delay Modifier's expiration period is 24 hours:** • ⏳ pending deployment. txExpiration() on the Delay Modifier must be 86,400 seconds (24 hours), as in P1.13.

Admin

P3.Admin: Only the Term multisig holders on the ProposerSafe may unpause the execution of Delay Modifier transactions.

#	Statement	Evidence
P3.Admin.1	Unpause Delay Modifier executions (PauseGuard.unpause()) is only callable by the PauseGuard admin, and not by the PauseGuard pauser.	**FV — unpause is only callable by the PauseGuard admin, and not by the PauseGuard pauser:** • DP-6 unpauseRevertsForAnyoneButTheAdmin ✅ passes: anyone who is not the admin is rejected. • DP-7 pauserAloneCannotUnpause ✅ passes: being the pauser, without being the admin, does not let a caller lift a pause. • DP-8 unpauseSucceedsForTheAdmin ✅ passes: from a paused state, the admin's unpause() succeeds and paused is then false. So DP-6 and DP-7 do not hold just because unpause always reverts.
P3.Admin.2	PauseGuard admin = ProposerSafe, holding: 1. the owners, fallbackHandler, modules, guard constant for the PauseSafe and ProposerSafe, and 2. the guard, txCooldown, txExpiration constant for the Delay Modifier, and 3. the pauser and admin constant for the PauseGuard.	**On-chain — The ProposerSafe is the PauseGuard's admin:** • ⏳ pending deployment. admin() on the PauseGuard must return the ProposerSafe. admin is immutable (PauseGuard.sol:17), so the PauseGuard's constructor sets it.
P3.Admin.3	Only Modules of ProposerSafe and Owners of ProposerSafe can interact with the ProposerSafe, except execTransaction, which anyone can call only if Owners of	Same as P1.9.

#	Statement	Evidence
	ProposerSafe have approved the transaction, and except `fallback` and `receive`, which anyone can call but which do not change state, assuming the fallbackHandler is `address(0)`.	
P3.Admin.4	ProposerSafe fallbackHandler is `address(0)`, holding: 1. the owners, fallbackHandler, modules, guard constant for the PauseSafe and ProposerSafe, and 2. the guard, txCooldown, txExpiration constant for the Delay Modifier, and 3. the pauser and admin constant for the PauseGuard.	Same as P1.10.
P3.Admin.5	`Owners of ProposerSafe = Term multisig holders on ProposerSafe`, holding: 1. the owners, fallbackHandler, modules, guard constant for the PauseSafe and ProposerSafe, and 2. the guard, txCooldown, txExpiration constant for the Delay Modifier, and 3. the pauser and admin constant for the PauseGuard.	Same as P1.11.
P3.Admin.6	`Modules of ProposerSafe = null`, holding: 1. the owners, fallbackHandler, modules, guard constant for the PauseSafe and ProposerSafe, and 2. the guard, txCooldown, txExpiration constant for the Delay Modifier, and 3. the pauser and admin constant for the PauseGuard.	Same as P1.12.
P3.Admin.7	The Term multisig holders on the ProposerSafe may unpause the execution of Delay Modifier transactions.	**FV — The ProposerSafe's owners can have it unpause the PauseGuard, and unpausing reopens Delay Modifier execution:** • DP-8 `unpauseSucceedsForTheAdmin` ✅ passes: from a paused state, the admin's `unpause()` succeeds and `paused` is then false. • SE150-6 `ownersCanMakeTheSafeAct` ✅ passes: once the threshold is passed, `execTransaction` succeeds and the Safe acts, for any threshold and any number of owners. So the ProposerSafe's owners (P3.Admin.5) can have it call `unpause` on the PauseGuard. • The ProposerSafe's guard is ConfigLockGuard (P5.ProposerSafe.5). Its call to `unpause` on the PauseGuard is a plain call to neither the ProposerSafe nor its locked Delay Modifier, so it is not a locked call. CL150-10 `checkTransactionAcceptsEverythingElse` ✅ passes: whatever `lockedDelay` is, the guard accepts every plain call that is not a locked call. It checks ConfigLockGuard's `checkTransaction` directly, so the Safe version does not matter. Its `checkAfterExecution` does nothing (`ConfigLockGuard.sol:141`), so the call is unchanged. • DP-17 `unpauseReopensExecuteNextTx` ✅ passes: with the PauseGuard installed and paused, after the admin unpauses, a following `executeNextTx` can succeed and reach the Delay Modifier's target. • DP-18 `checkTransactionAcceptsWhileNotPaused` ✅ passes: while not paused, the PauseGuard accepts any transaction from any caller, so it adds no revert of its own to `executeNextTx`.

#	Statement	Evidence
		On-chain — The PauseGuard is the Delay Modifier's guard: • ⏳ pending deployment. `guard()` on the Delay Modifier must return the PauseGuard, as in P1.13.

Proposition 4 — The following cannot be changed by anyone:

1. the owner, target, modules, guard, role targets, role scopes and role assignments for the Roles Modifier
2. the voting token, proposal threshold, voting threshold and quorum for the Governor

---

Roles Modifier

P4.RolesModifier: The owner, target, modules, guard, role targets, role scopes and role assignments for the Roles Modifier cannot be changed by anyone.

#	Statement	Evidence
P4.RolesModifier.1	Only Roles Modifier Modules with AssignedRole or Roles Modifier Owner can interact with Roles Modifier.	Same as P2.8.
P4.RolesModifier.2	The owner, target, modules, guard, role targets, role scopes and role assignments for the Roles Modifier can only be changed by the Roles Modifier Owner, and not by Roles Modifier Modules with AssignedRole.	**FV — The owner, target, modules, guard, role targets, role scopes and role assignments for the Roles Modifier can only be changed by the Roles Modifier Owner, not by Roles Modifier Modules with AssignedRole:** • RI-2 `rolesConfigOnlyChangesThroughOwner` ✅ passes: over every write function except `setUp`, the owner, target, module list, guard, role membership, default roles and target clearances change only when the caller is the Roles Modifier Owner. • RI-1 `setUpAlwaysRevertsAfterDeployment` ✅ passes: `setUp`, the one write function RI-2 skips, always reverts, so it cannot reset these settings. • RI-5 `onlyOwnerCanCallRolesSettings` ✅ passes: RI-2 does not track function and parameter scoping, so RI-5 covers it by call: `transferOwnership`, `renounceOwnership`, `setTarget`, `enableModule`, `disableModule`, `setGuard`, `allowTarget`, `revokeTarget`, `scopeTarget`, `scopeAllowFunction`, `scopeRevokeFunction`, `scopeFunction`, `scopeFunctionExecutionOptions`, `scopeParameter`, `scopeParameterAsOneOf`, `unscopeParameter`, `assignRoles` and `setDefaultRole` succeed only when the caller is the Roles Modifier Owner. • By P2.9, the Roles Modifier Owner is not a module, so no Roles Modifier Module with AssignedRole can change these settings.
P4.RolesModifier.3	`Roles Modifier Owner = address(0)`, which cannot be changed by anyone.	**On-chain — Roles Modifier Owner is `address(0)`:** • ⏳ pending deployment. `owner()` on the Roles Modifier must return `address(0)`, and its `_owner` slot (slot 51, `OwnableUpgradeable`) must hold zero: whoever deploys and configures the Roles Modifier must renounce ownership once Role 1 is set up. **FV — Roles Modifier Owner cannot change from `address(0)`:** • RI-16 `noOwnerStaysNoOwner` ✅ passes: with the owner (Roles Modifier Owner) at `address(0)`, no function other than `setUp` can give the Roles Modifier an owner again, including `transferOwnership`. • RI-1 `setUpAlwaysRevertsAfterDeployment` ✅ passes: `setUp`, the one function RI-16 skips, always reverts, so it cannot set a new owner.

#	Statement	Evidence
P4.RolesModifier.4	The owner, target, modules, guard, role targets, role scopes and role assignments for the Roles Modifier cannot be changed by anyone because `Roles Modifier Owner = address(0)`.	**FV — With the owner at `address(0)`, none of these settings can be changed by anyone:** • By P4.RolesModifier.2, only the Roles Modifier Owner can change these settings, and by P4.RolesModifier.3, the owner is `address(0)` for good. • RI-15 `noOwnerSettingsAlwaysRevert` ✅ passes: with the owner (Roles Modifier Owner) at `address(0)`, `transferOwnership`, `renounceOwnership`, `setTarget`, `enableModule`, `disableModule`, `setGuard`, `allowTarget`, `revokeTarget`, `scopeTarget`, `scopeAllowFunction`, `scopeRevokeFunction`, `scopeFunction`, `scopeFunctionExecutionOptions`, `scopeParameter`, `scopeParameterAsOneOf`, `unscopeParameter`, `assignRoles` and `setDefaultRole` revert for every caller except `address(0)`, which never sends a transaction.

Governor

P4.Governor: The voting token, proposal threshold, voting threshold and quorum for the Governor cannot be changed by anyone.

#	Statement	Evidence
P4.Governor.1	Only Holders of Governor Vote Token can interact with the Governor, except `execute`, which anyone can call only if Holders of Governor Vote Token proposed and passed the proposal, and except the `castVote` functions (`castVote`, `castVoteWithReason`, `castVoteWithReasonAndParams`, `castVoteBySig`, `castVoteWithReasonAndParamsBySig`), which anyone can call but which count a non-holder's vote as 0, and except `receive`, `onERC721Received`, `onERC1155Received` and `onERC1155BatchReceived`, which anyone can call but which make no storage writes.	Same as P2.12.
P4.Governor.2	The voting token, proposal threshold, voting threshold and quorum for the Governor cannot be changed by Holders of Governor Vote Token, so they cannot be changed by anyone.	**FV — The voting token, proposal threshold, voting threshold and quorum for the Governor cannot be changed by Holders of Governor Vote Token, so they cannot be changed by anyone:** • GS-1 `governorWriteFunctionsAreTheKnownFourteen` ✅ passes: the Governor's write functions are exactly the fourteen listed in P2.12. • GS-2 `governorSettingsNeverChange` ✅ passes: no write function changes the voting token, proposal threshold or quorum, whoever calls it and with whatever arguments. This includes `relay` called by the Governor as an action of a passed proposal. • GS-4 `queueAlwaysReverts` ✅ passes: `queue`, the one write function GS-2 skips, never succeeds. So none of the fourteen changes the voting token, proposal threshold or quorum, whoever calls it. • GS-3 `governorSettingsAreTheDeployedValues` ✅ passes: the proposal threshold (1,000 TERM) and the quorum (1,000,000 TERM, the constant `QUORUM`, at every timepoint) are fixed in the Governor's code, and the voting token is an immutable, so nothing the Governor stores can change them. • GS-16 `votingThresholdIsTheDeployedRule` ✅ passes: the voting threshold is fixed in the Governor's code

#	Statement	Evidence
		(`GovernorCountingSimple.sol:61`). In every state, a proposal passes the vote exactly when its For votes are strictly more than its Against votes. The only stored data it reads is the tallies, and only votes move them (GC-1 to GC-4, P2.12).

Proposition 5 — The owners for the ProposerSafe, OwnerlessSafe, and DelayOwnerSafe can be changed only by the Term multisig holders on the ProposerSafe, OwnerlessSafe, or DelayOwnerSafe. The fallbackHandler, modules, guard, module guard for the OwnerlessSafe can be changed only by the Term multisig holders on the ProposerSafe. The following: 1. the fallbackHandler, modules, guard for the ProposerSafe and DelayOwnerSafe, 2. the module guard for the DelayOwnerSafe, and 3. the owner, target, modules, guard, txCooldown, txExpiration for the Delay Modifier cannot be changed by anyone.

ProposerSafe

P5.ProposerSafe: The owners for the ProposerSafe can only be changed by Term multisig holders on the ProposerSafe; the fallbackHandler, modules, guard for the ProposerSafe cannot be changed by anyone.

#	Statement	Evidence
P5.ProposerSafe.1	Only Modules of ProposerSafe and Owners of ProposerSafe can interact with the ProposerSafe, except `execTransaction`, which anyone can call only if Owners of ProposerSafe have approved the transaction, and except `fallback` and `receive`, which anyone can call but which do not change state, assuming the fallbackHandler is `address(0)`.	Same as P1.9.
P5.ProposerSafe.2	ProposerSafe fallbackHandler is `address(0)`, which can only be changed by Owners of ProposerSafe and Modules of ProposerSafe.	**On-chain — ProposerSafe fallbackHandler is `address(0)`:** • Same as P1.10. **FV — ProposerSafe fallbackHandler can only be changed by Owners of ProposerSafe or Modules of ProposerSafe:** • SC150-4 `settingsOnlyChangeThroughAModuleOrOwners` ✅ passes: over every write function, `fallback` included while no fallbackHandler is set (P1.10), the fallbackHandler only changes through an enabled module's `execTransactionFromModule` or `execTransactionFromModuleReturnData`, or through `execTransaction`. • By P1.9, only Modules of ProposerSafe can call `execTransactionFromModule` and `execTransactionFromModuleReturnData` (SE150-3), and `execTransaction` only acts once as many Owners of ProposerSafe as the threshold have approved (SE150-5, SE150-12). So only Modules of ProposerSafe or Owners of ProposerSafe can change the fallbackHandler.
P5.ProposerSafe.3	`Modules of ProposerSafe = null`, which can only be changed by Owners of ProposerSafe because `Modules of ProposerSafe = null`.	**On-chain — The Modules of ProposerSafe list is empty:** • Same as P1.12. **FV — Modules of ProposerSafe can only be changed by Owners of ProposerSafe because `Modules of ProposerSafe = null`:**

#	Statement	Evidence
		• SC150-4 `settingsOnlyChangeThroughAModuleOrOwners` ✅ passes: over every write function, `fallback` included while no fallbackHandler is set (P1.10), the module list only changes through an enabled module's `execTransactionFromModule` or `execTransactionFromModuleReturnData`, or through `execTransaction`. • By P1.9, only Modules of ProposerSafe can call `execTransactionFromModule` and `execTransactionFromModuleReturnData` (SE150-3), and `execTransaction` only acts once as many Owners of ProposerSafe as the threshold have approved (SE150-5, SE150-12). • The ProposerSafe has no module (P1.12), so only the Owners of ProposerSafe can change the module list.
P5.ProposerSafe.4	`Owners of ProposerSafe = Term multisig holders on the ProposerSafe`, which can only be changed by Term multisig holders on the ProposerSafe because `Modules of ProposerSafe = null`.	**On-chain — Owners of ProposerSafe are these eleven Term multisig holders, and five of them must approve each transaction:** • Same as P1.11. **FV — Owners of ProposerSafe can only be changed by Term multisig holders on the ProposerSafe because `Modules of ProposerSafe = null`:** • SC150-4 `settingsOnlyChangeThroughAModuleOrOwners` ✅ passes: over every write function, `fallback` included while no fallbackHandler is set (P1.10), the owners and threshold only change through an enabled module's `execTransactionFromModule` or `execTransactionFromModuleReturnData`, or through `execTransaction`. • By P1.9, only Modules of ProposerSafe can call `execTransactionFromModule` and `execTransactionFromModuleReturnData` (SE150-3), and `execTransaction` only acts once five Owners of ProposerSafe, the Term multisig holders (P1.11), have approved (SE150-5, SE150-12). • The ProposerSafe has no module (P1.12), so only the Term multisig holders on the ProposerSafe can change the owners and threshold.
P5.ProposerSafe.5	The owners for the ProposerSafe can be changed by Term multisig holders on the ProposerSafe; the fallbackHandler, modules, guard for the ProposerSafe cannot be changed by Term multisig holders on the ProposerSafe, so they cannot be changed by anyone.	**FV — The Term multisig holders on the ProposerSafe can change Owners of ProposerSafe through `execTransaction`:** • CL150-21 `guardedOwnersCanStillChangeOtherSettings` ✅ passes, for each of `addOwnerWithThreshold`, `removeOwner`, `swapOwner` and `changeThreshold`: with ConfigLockGuard installed, an `execTransaction` addressed to the Safe with a call to that function succeeds and the owners or threshold change. • CL150-21 stubs the signature check. By P1.9, `execTransaction` only acts once five Owners of ProposerSafe, the Term multisig holders (P1.11), have approved (SE150-5, SE150-12), and their approvals pass it (SE150-6). **FV — The Term multisig holders on the ProposerSafe cannot change ProposerSafe fallbackHandler, Modules of ProposerSafe or ProposerSafe guard through `execTransaction` if ProposerSafe guard is ConfigLockGuard, so nobody can change them:** • CL150-2 `checkTransactionRejectsSelfSetGuard`, CL150-3 `checkTransactionRejectsSelfEnableModule`, CL150-4 `checkTransactionRejectsSelfDisableModule`, CL150-5 `checkTransactionRejectsSelfSetFallbackHandler` ✅ pass: called by the Safe, the guard reverts on any transaction to the Safe itself carrying `setGuard`, `enableModule`, `disableModule` or `setFallbackHandler`. • CL150-12 `guardedSafeCannotCallSetGuard`, CL150-13 `guardedSafeCannotCallEnableModule`, CL150-14 `guardedSafeCannotCallDisableModule`, CL150-15 `guardedSafeCannotCallSetFallbackHandler` ✅ pass: with ConfigLockGuard installed, an `execTransaction` addressed to the Safe with any of these calls reverts. • CL150-17 `guardedSelfCallNeverChangesGuard`, CL150-18 `guardedSelfCallNeverChangesModules`, CL150-19 `guardedSelfCallNeverChangesFallbackHandler` ✅ pass: for any calldata and operation, an `execTransaction` addressed to the Safe leaves the guard, every entry of the Modules of ProposerSafe list and the fallbackHandler unchanged. • CL150-1 `checkTransactionRejectsDelegateCall`, CL150-16 `guardedSafeCannotDelegateCall` ✅ pass: the guard rejects every delegate call, so no delegated code can write the guard, the Modules of ProposerSafe list or the

#	Statement	Evidence
		fallbackHandler directly. • By P1.9, these settings only change when the caller is the Safe itself (SE150-7), so an `execTransaction` to any other address cannot change them either. • These rules leave `lockedDelay` unconstrained, so they hold whichever modifier ConfigLockGuard was built with. • By P5.ProposerSafe.2 and P5.ProposerSafe.3, only Owners of ProposerSafe or Modules of ProposerSafe can change the fallbackHandler and the Modules of ProposerSafe list, and SC150-4 (P5.ProposerSafe.2) covers the guard the same way. The ProposerSafe has no module (P5.ProposerSafe.3), so nobody can change them. **On-chain — ProposerSafe guard is ConfigLockGuard:** • ⏳ pending deployment. The guard slot (`0x4a204f620c8c5ccdca3fd54d003badd85ba500436a431f0cbda4f558c93c34c8`, `GuardManager.sol:66` in Safe v1.5.0) on the ProposerSafe must hold ConfigLockGuard. • Assumes the ProposerSafe runs Safe v1.5.0, the version CL150 checks (`GuardManager.sol:66`).

DelayOwnerSafe

P5.DelayOwnerSafe: The owners for the DelayOwnerSafe can only be changed by Term multisig holders on the DelayOwnerSafe; the fallbackHandler, modules, guard, module guard for the DelayOwnerSafe cannot be changed by anyone.

#	Statement	Evidence
P5.DelayOwnerSafe.1	Only Modules of DelayOwnerSafe and Owners of DelayOwnerSafe can interact with the DelayOwnerSafe, except `execTransaction`, which anyone can call only if Owners of DelayOwnerSafe have approved the transaction, and except `fallback` and `receive`, which anyone can call but which do not change state, assuming the fallbackHandler is `address(0)`.	Same as P2.4.
P5.DelayOwnerSafe.2	DelayOwnerSafe fallbackHandler is `address(0)`, which can only be changed by Owners of DelayOwnerSafe and Modules of DelayOwnerSafe.	**On-chain — DelayOwnerSafe fallbackHandler is `address(0)`:** • Same as P2.5. **FV — DelayOwnerSafe fallbackHandler can only be changed by Owners of DelayOwnerSafe or Modules of DelayOwnerSafe:** • SC150-4 `settingsOnlyChangeThroughAModuleOrOwners` ✅ passes: over every write function, `fallback` included while no fallbackHandler is set (P2.5), the fallbackHandler only changes through an enabled module's `execTransactionFromModule` or `execTransactionFromModuleReturnData`, or through `execTransaction`. • By P2.4, only Modules of DelayOwnerSafe can call `execTransactionFromModule` and `execTransactionFromModuleReturnData` (SE150-3), and `execTransaction` only acts once as many Owners of DelayOwnerSafe as the threshold have approved (SE150-5, SE150-12). So only Modules of DelayOwnerSafe or Owners of DelayOwnerSafe can change the fallbackHandler.

#	Statement	Evidence
P5.DelayOwnerSafe.3	`Owners of DelayOwnerSafe = Term multisig holders on the DelayOwnerSafe`, which can only be changed by Term multisig holders on the DelayOwnerSafe and Modules of DelayOwnerSafe.	**On-chain — Owners of DelayOwnerSafe are these eleven Term multisig holders, and five of them must approve each transaction:** • Same as P2.6. **FV — Owners of DelayOwnerSafe can only be changed by Term multisig holders on the DelayOwnerSafe or Modules of DelayOwnerSafe:** • SC150-4 `settingsOnlyChangeThroughAModuleOrOwners` ✅ passes: over every write function, `fallback` included while no fallbackHandler is set (P2.5), the owners and threshold only change through an enabled module's `execTransactionFromModule` or `execTransactionFromModuleReturnData`, or through `execTransaction`. • By P2.4, only Modules of DelayOwnerSafe can call `execTransactionFromModule` and `execTransactionFromModuleReturnData` (SE150-3), and `execTransaction` only acts once five Owners of DelayOwnerSafe, the Term multisig holders (P2.6), have approved (SE150-5, SE150-12). So only Modules of DelayOwnerSafe or the Term multisig holders on the DelayOwnerSafe can change the owners and threshold.
P5.DelayOwnerSafe.4	`Modules of DelayOwnerSafe = Roles Modifier`, which can only be changed by Term multisig holders on the DelayOwnerSafe and Roles Modifier.	**On-chain — The Modules of DelayOwnerSafe list contains only the Roles Modifier:** • Same as P2.7. **FV — Modules of DelayOwnerSafe can only be changed by Term multisig holders on the DelayOwnerSafe or Roles Modifier:** • SC150-4 `settingsOnlyChangeThroughAModuleOrOwners` ✅ passes: over every write function, `fallback` included while no fallbackHandler is set (P2.5), the module list only changes through an enabled module's `execTransactionFromModule` or `execTransactionFromModuleReturnData`, or through `execTransaction`. • By P2.4, only Modules of DelayOwnerSafe can call `execTransactionFromModule` and `execTransactionFromModuleReturnData` (SE150-3), and `execTransaction` only acts once five Owners of DelayOwnerSafe, the Term multisig holders (P2.6), have approved (SE150-5, SE150-12). • The DelayOwnerSafe's only module is the Roles Modifier (P2.7), so only the Roles Modifier or the Term multisig holders on the DelayOwnerSafe can change the module list.
P5.DelayOwnerSafe.5	Only Roles Modifier Modules with AssignedRole or Roles Modifier Owner can interact with Roles Modifier.	Same as P2.8.
P5.DelayOwnerSafe.6	`Roles Modifier Owner = address(0)`, which cannot be changed, P4.RolesModifier.	Same as P4.RolesModifier.3.
P5.DelayOwnerSafe.7	`Roles Modifier Modules with AssignedRole = Governor`, which cannot be changed by anyone, P4.RolesModifier.	Same as P2.10. See P4.RolesModifier: the Roles Modifier's modules and role assignments cannot be changed by anyone.
P5.DelayOwnerSafe.8	Governor is assigned only Role 1, which cannot be changed by anyone, P4.RolesModifier.	Same as P2.11. See P4.RolesModifier: the Governor's role assignments and default role cannot be changed by anyone.
P5.DelayOwnerSafe.9	Role 1 and the SetTxNonceGuard together prevent Governor from changing the owners, fallbackHandler, modules, guard, module guard for the DelayOwnerSafe.	**FV — Role 1 and the SetTxNonceGuard together restrict the Governor to vetoes (`Delay.setTxNonce(uint256)`), so the Governor cannot change the owners, fallbackHandler, modules, guard, module guard for the DelayOwnerSafe:** • RG-2 `governorExecTransactionWithRoleLimitedToDelaySetTxNonce`, RG-3 `governorExecTransactionFromModuleLimitedToDelaySetTxNonce`, RG-4 `governorExecTransactionWithRoleReturnDataLimitedToDelaySetTxNonce`, RG-5

#	Statement	Evidence
	SetTxNonceGuard cannot be changed by anyone, P4.RolesModifier.	`governorExecTransactionFromModuleReturnDataLimitedToDelaySetTxNonce` ✅ pass: every call the Governor successfully executes through the Roles Modifier is `setTxNonce(uint256)` on the Delay Modifier, with zero value and as a plain `Call`, on all four execution entry points: `execTransactionWithRole`, `execTransactionFromModule`, `execTransactionWithRoleReturnData` and `execTransactionFromModuleReturnData`. • The three on-chain blobs below give the setup these rules assume: the SetTxNonceGuard installed and pointed at the Delay Modifier; the Governor an enabled module (P2.10), a member of Role 1 only, with default Role 1; and Role 1's target clearances, Function on the Delay Modifier and None on every other target. • So through its module, the Roles Modifier, the Governor can only make the DelayOwnerSafe call the Delay Modifier's `setTxNonce`. The DelayOwnerSafe's settings only change when it calls itself (SE150-7, P2.4), so the Governor cannot change its owners, fallbackHandler, modules, guard or module guard. **On-chain — The SetTxNonceGuard is the Roles Modifier's guard and is pointed at the Delay Modifier:** • ⏳ pending deployment. `guard()` on the Roles Modifier must return the SetTxNonceGuard, and `getGuard()` must return the same address. • ⏳ pending deployment. `delay()` on the SetTxNonceGuard must return the Delay Modifier, and `target()` on the Roles Modifier must return the DelayOwnerSafe, not the Roles Modifier itself. **On-chain — Governor is assigned only Role 1:** • Same as P5.DelayOwnerSafe.8. **On-chain — Role 1 has Function clearance on the Delay Modifier and no clearance on any other target:** • ⏳ pending deployment. The Roles Modifier's complete event history, from its creation, must have exactly one `ScopeTarget`: Role 1, the Delay Modifier, and no `AllowTarget` or `RevokeTarget`. Clearances start at None, and only `allowTarget`, `scopeTarget` and `revokeTarget` set them, each emitting its event (`Permissions.sol:385`, `:397`, `:409`). So Role 1 has Function clearance on the Delay Modifier and None on every other target. • These clearances cannot change: the Roles Modifier must have no owner (P4.RolesModifier.3), and by RI-2 only the owner can change them. **Already proven — SetTxNonceGuard cannot be changed by anyone:** • See P4.RolesModifier: the Roles Modifier's guard, the SetTxNonceGuard, cannot be changed by anyone. The guard's `delay` is `immutable` (`SetTxNonceGuard.sol:14`).
P5.DelayOwnerSafe.10	The owners for the DelayOwnerSafe can be changed by Term multisig holders on the DelayOwnerSafe; the fallbackHandler, modules, guard, module guard for the DelayOwnerSafe cannot be changed by Term multisig holders on the DelayOwnerSafe, so they cannot be changed by anyone.	**FV — The Term multisig holders on the DelayOwnerSafe can change Owners of DelayOwnerSafe through `execTransaction`:** • CL150-21 `guardedOwnersCanStillChangeOtherSettings` ✅ passes on Safe v1.5.0, the DelayOwnerSafe's code (P2.4), for each of `addOwnerWithThreshold`, `removeOwner`, `swapOwner` and `changeThreshold`: with ConfigLockGuard installed, an `execTransaction` addressed to the Safe with a call to that function succeeds and the owners or threshold change. • CL150-21 stubs the signature check. By P2.4, `execTransaction` only acts once five Owners of DelayOwnerSafe, the Term multisig holders (P2.6), have approved (SE150-5, SE150-12), and their approvals pass it (SE150-6). **FV — The Term multisig holders on the DelayOwnerSafe cannot change DelayOwnerSafe fallbackHandler, Modules of DelayOwnerSafe, DelayOwnerSafe guard or DelayOwnerSafe module guard through `execTransaction` if DelayOwnerSafe guard is ConfigLockGuard, so nobody can change them:** • CL150-2 `checkTransactionRejectsSelfSetGuard`, CL150-3 `checkTransactionRejectsSelfEnableModule`, CL150-4 `checkTransactionRejectsSelfDisableModule`, CL150-5 `checkTransactionRejectsSelfSetFallbackHandler` ✅ pass: called by the Safe, the guard reverts on any transaction to the Safe itself carrying `setGuard`, `enableModule`,

#	Statement	Evidence
		`disableModule` or `setFallbackHandler`. • CL150-12 `guardedSafeCannotCallSetGuard`, CL150-13 `guardedSafeCannotCallEnableModule`, CL150-14 `guardedSafeCannotCallDisableModule`, CL150-15 `guardedSafeCannotCallSetFallbackHandler` ✅ pass: with ConfigLockGuard installed, an `execTransaction` addressed to the Safe with any of these calls reverts. • CL150-17 `guardedSelfCallNeverChangesGuard`, CL150-18 `guardedSelfCallNeverChangesModules`, CL150-19 `guardedSelfCallNeverChangesFallbackHandler` ✅ pass: for any calldata and operation, an `execTransaction` addressed to the Safe leaves the guard, every entry of the Modules of DelayOwnerSafe list and the fallbackHandler unchanged. • CL150-25 `checkTransactionRejectsSelfSetModuleGuard`, CL150-26 `guardedSafeCannotCallSetModuleGuard`, CL150-27 `guardedSelfCallNeverChangesModuleGuard` ✅ pass: called by the Safe, the guard reverts on any transaction to the Safe itself carrying `setModuleGuard`; with ConfigLockGuard installed, an `execTransaction` addressed to the Safe with that call reverts; and for any calldata and operation, an `execTransaction` addressed to the Safe leaves the module guard unchanged. • CL150-1 `checkTransactionRejectsDelegateCall`, CL150-16 `guardedSafeCannotDelegateCall` ✅ pass: the guard rejects every delegate call, so no delegated code can write the guard, the Modules of DelayOwnerSafe list, the fallbackHandler or the module guard directly. • By P2.4, these settings only change when the caller is the Safe itself (SE150-7), so an `execTransaction` to any other address cannot change them either. • These rules run on Safe v1.5.0, the DelayOwnerSafe's code (P2.4), and leave `lockedDelay` unconstrained, so they hold whichever modifier ConfigLockGuard was built with. • By P5.DelayOwnerSafe.2 and P5.DelayOwnerSafe.4, only Owners of DelayOwnerSafe or Modules of DelayOwnerSafe can change the fallbackHandler and the Modules of DelayOwnerSafe list, and SC150-4 (P5.DelayOwnerSafe.2) covers the guard and the module guard the same way. The only Module of DelayOwnerSafe is the Roles Modifier (P5.DelayOwnerSafe.4), and by P5.DelayOwnerSafe.5 to P5.DelayOwnerSafe.9 nobody can change them through it, so nobody can change them. **On-chain — DelayOwnerSafe guard is ConfigLockGuard:** • ⏳ pending deployment. The guard slot (`0x4a204f620c8c5ccdca3fd54d003badd85ba500436a431f0cbda4f558c93c34c8`, `GuardManager.sol:66` in Safe v1.5.0) on the DelayOwnerSafe must hold ConfigLockGuard. • Assumes the DelayOwnerSafe runs Safe v1.5.0, the version CL150 checks (`GuardManager.sol:66`).

Delay Modifier

P5.DelayModifier: The owner, target, modules, guard, txCooldown, txExpiration for the Delay Modifier cannot be changed by anyone.

#	Statement	Evidence
P5.DelayModifier.1	Only Modules of Delay Modifier or Owner of Delay Modifier can interact with Delay Modifier, except `executeNextTx` and `skipExpired`, which anyone can call, but they can only execute or skip transactions queued by Modules of Delay Modifier or Owner of Delay Modifier.	Same as P1.6.

#	Statement	Evidence
P5.DelayModifier.2	The owner, target, modules, guard, txCooldown, txExpiration for the Delay Modifier can only be changed by Owner of Delay Modifier, not by Modules of Delay Modifier.	**FV — The owner, target, modules, guard, txCooldown, txExpiration for the Delay Modifier can only be changed by Owner of Delay Modifier, not Modules of Delay Modifier:** • DM-8 `atMostOneCallerPassesOnlyOwner` ✅ passes: each of the Delay Modifier's ten `onlyOwner` functions, which include `setTxCooldown`, `setTxExpiration`, `setTarget`, `enableModule`, `disableModule`, `setGuard`, `transferOwnership` and `renounceOwnership`, succeeds only when the caller is the owner (Owner of Delay Modifier). • DM-6 `ownerOnlyChangesThroughOwnableTransfer`, DM-2 `modulesOnlyChangeThroughOwnerEnableOrDisable`, DM-7 `guardOnlyChangesThroughOwnerSetGuard` ✅ pass: over every write function, the owner, the module list and the guard change only when the caller is the owner. • By P1.7, the owner is not a module, so no Module of Delay Modifier can change these settings.
P5.DelayModifier.3	`Owner of Delay Modifier = DelayOwnerSafe`, which can only be changed by the DelayOwnerSafe.	**On-chain — Owner is DelayOwnerSafe:** • Same as P2.3. **FV — Owner of Delay Modifier can only be changed by Owner of Delay Modifier:** • DM-6 `ownerOnlyChangesThroughOwnableTransfer` ✅ passes: over every write function, the owner changes only through `transferOwnership` or `renounceOwnership`, and only when the caller is the current owner (Owner of Delay Modifier). • So, since the owner is the DelayOwnerSafe, only the DelayOwnerSafe can change it.
P5.DelayModifier.4	Only Modules of DelayOwnerSafe and Owners of DelayOwnerSafe can interact with the DelayOwnerSafe, except `execTransaction`, which anyone can call only if Owners of DelayOwnerSafe have approved the transaction, and except `fallback` and `receive`, which anyone can call but which do not change state, assuming the fallbackHandler is `address(0)`.	Same as P2.4.
P5.DelayModifier.5	DelayOwnerSafe fallbackHandler is `address(0)`, which cannot be changed by anyone, P5.DelayOwnerSafe.	**Already proven — DelayOwnerSafe fallbackHandler is `address(0)`:** • Same as P2.5. **Already proven — DelayOwnerSafe fallbackHandler cannot be changed by anyone:** • See P5.DelayOwnerSafe: by P5.DelayOwnerSafe.10, the Term multisig holders on the DelayOwnerSafe cannot change the fallbackHandler, so nobody can.
P5.DelayModifier.6	`Owners of DelayOwnerSafe = Term multisig holders on the DelayOwnerSafe`, which can only be changed by Term multisig holders on the DelayOwnerSafe, P5.DelayOwnerSafe.	**Already proven — Owners of DelayOwnerSafe are these eleven Term multisig holders, and five of them must approve each transaction:** • Same as P2.6. **Already proven — Owners of DelayOwnerSafe can only be changed by Term multisig holders on the DelayOwnerSafe:** • See P5.DelayOwnerSafe.
P5.DelayModifier.7	`Modules of DelayOwnerSafe = Roles Modifier`, which cannot be changed by anyone, P5.DelayOwnerSafe.	**Already proven — The Modules of DelayOwnerSafe list contains only the Roles Modifier:** • Same as P2.7.

#	Statement	Evidence
		Already proven — Modules of DelayOwnerSafe cannot be changed by anyone: • See P5.DelayOwnerSafe: by P5.DelayOwnerSafe.10, the Term multisig holders on the DelayOwnerSafe cannot change the Modules of DelayOwnerSafe list, so nobody can.
P5.DelayModifier.8	Only Roles Modifier Modules with AssignedRole or Roles Modifier Owner can interact with Roles Modifier.	Same as P2.8.
P5.DelayModifier.9	`Roles Modifier Owner = address(0)`, which cannot be changed, P4.RolesModifier.	Same as P4.RolesModifier.3.
P5.DelayModifier.10	`Roles Modifier Modules with AssignedRole = Governor`, which cannot be changed by anyone, P4.RolesModifier.	Same as P5.DelayOwnerSafe.7.
P5.DelayModifier.11	Governor is assigned only Role 1, which cannot be changed by anyone, P4.RolesModifier.	Same as P5.DelayOwnerSafe.8.
P5.DelayModifier.12	Role 1 and the SetTxNonceGuard together prevent Governor from changing the owner, target, modules, guard, txCooldown, txExpiration for the Delay Modifier. SetTxNonceGuard cannot be changed by anyone, P4.RolesModifier.	**FV — Role 1 and the SetTxNonceGuard together restrict the Governor to vetoes (`Delay.setTxNonce(uint256)`), so the Governor cannot change the owner, target, modules, guard, txCooldown, txExpiration for the Delay Modifier:** • RG-2 `governorExecTransactionWithRoleLimitedToDelaySetTxNonce`, RG-3 `governorExecTransactionFromModuleLimitedToDelaySetTxNonce`, RG-4 `governorExecTransactionWithRoleReturnDataLimitedToDelaySetTxNonce`, RG-5 `governorExecTransactionFromModuleReturnDataLimitedToDelaySetTxNonce` ✅ pass: every call the Governor successfully executes through the Roles Modifier is `setTxNonce(uint256)` on the Delay Modifier, with zero value and as a plain `Call`, on all four execution entry points: `execTransactionWithRole`, `execTransactionFromModule`, `execTransactionWithRoleReturnData` and `execTransactionFromModuleReturnData`. • The three on-chain blobs below give the setup these rules assume: the SetTxNonceGuard installed and pointed at the Delay Modifier; the Governor an enabled module (P2.10), a member of Role 1 only, with default Role 1; and Role 1's target clearances, Function on the Delay Modifier and None on every other target. • So through the Roles Modifier, the Governor can only make the DelayOwnerSafe call the Delay Modifier's `setTxNonce`, which only writes `txNonce` (`Delay.sol:`136–143). So the Governor cannot change the Delay Modifier's owner, target, modules, guard, txCooldown or txExpiration. **On-chain — The SetTxNonceGuard is the Roles Modifier's guard and is pointed at the Delay Modifier:** • ⏳ pending deployment. `guard()` on the Roles Modifier must return the SetTxNonceGuard, and `getGuard()` must return the same address. • ⏳ pending deployment. `delay()` on the SetTxNonceGuard must return the Delay Modifier, and `target()` on the Roles Modifier must return the DelayOwnerSafe, not the Roles Modifier itself. **On-chain — Governor is assigned only Role 1:** • Same as P5.DelayModifier.11. **On-chain — Role 1 has Function clearance on the Delay Modifier and no clearance on any other target:** • Same as P5.DelayOwnerSafe.9. **Already proven — SetTxNonceGuard cannot be changed by anyone:**

#	Statement	Evidence
		• See P4.RolesModifier: the Roles Modifier's guard, the SetTxNonceGuard, cannot be changed by anyone. The guard's `delay` is `immutable` (`SetTxNonceGuard.sol:14`).
P5.DelayModifier.13	The owner, target, modules, guard, txCooldown, txExpiration for the Delay Modifier cannot be changed by Term multisig holders on the DelayOwnerSafe, so they cannot be changed by anyone.	**FV — The Term multisig holders on the DelayOwnerSafe cannot change the owner, target, modules, guard, txCooldown or txExpiration of the Delay Modifier through `execTransaction` if DelayOwnerSafe guard is ConfigLockGuard, so nobody can change them:** • CL150-6 `checkTransactionRejectsModifierOwnershipChange`, CL150-7 `checkTransactionRejectsModifierModuleChange`, CL150-8 `checkTransactionRejectsModifierSetGuard`, CL150-9 `checkTransactionRejectsModifierSetAvatarOrTarget` ✅ pass: called by the Safe, the guard reverts on any transaction to its locked modifier carrying `transferOwnership`, `renounceOwnership`, `enableModule`, `disableModule`, `setGuard`, `setAvatar` or `setTarget`. • CL150-24 `checkTransactionRejectsModifierTimingChange` ✅ passes: called by the Safe, the guard reverts on any transaction to its locked modifier carrying `setTxCooldown` or `setTxExpiration`. • CL150-23 `guardedSafeCannotCallLockedModifierFunctions` ✅ passes: with ConfigLockGuard installed, an `execTransaction` addressed to the locked modifier with any of these calls reverts, whatever the arguments, value and operation. • CL150-1 `checkTransactionRejectsDelegateCall`, CL150-16 `guardedSafeCannotDelegateCall` ✅ pass: the guard rejects every delegate call, so every `execTransaction` to the Delay Modifier that succeeds is a plain call carrying some other selector. • OD150-1 `unlockedDelayCallsNeverChangeOwnerGuardAvatarOrTarget`, OD150-2 `unlockedDelayCallsNeverChangeModules` ✅ pass: over every Delay Modifier write function except those nine and `setUp`, and for any caller, the owner, guard, target and module list do not change. • OD150-5 `unlockedDelayCallsNeverChangeTxCooldownOrTxExpiration` ✅ passes: over the same functions and any caller, txCooldown and txExpiration do not change. • An `execTransaction` to any other address reaches the Delay Modifier, if at all, with that address as the caller, not the owner, so it cannot change these settings (P5.DelayModifier.2). • The CL150 rules leave `lockedDelay` unconstrained, so they lock whichever modifier ConfigLockGuard was built with. The deployed guard's locked modifier is the Delay Modifier (below). • By P5.DelayModifier.2 and P5.DelayModifier.3, only the DelayOwnerSafe can change the owner, target, modules, guard, txCooldown and txExpiration. The DelayOwnerSafe acts only through `execTransaction` or its modules (P5.DelayModifier.4, P5.DelayModifier.5). Its only module is the Roles Modifier (P5.DelayModifier.7), through which nobody can change them (P5.DelayModifier.8 to P5.DelayModifier.12), so nobody can change them. **On-chain — DelayOwnerSafe guard is ConfigLockGuard, with the Delay Modifier as its locked modifier:** • Same as P5.DelayOwnerSafe.10: the DelayOwnerSafe's guard slot must hold ConfigLockGuard. • ⏳ pending deployment. `lockedDelay()` on ConfigLockGuard must return the Delay Modifier. • Assumes the DelayOwnerSafe runs Safe v1.5.0, the version CL150 and OD150 check.

OwnerlessSafe

P5.OwnerlessSafe: The owners for the OwnerlessSafe can only be changed by Term multisig holders on the ProposerSafe and Term multisig holders on the OwnerlessSafe; the fallbackHandler, modules, guard, module guard for the OwnerlessSafe can only be changed by Term multisig holders on the ProposerSafe.

#	Statement	Evidence
P5.OwnerlessSafe.1	Only Modules of OwnerlessSafe and Owners of OwnerlessSafe can interact with the OwnerlessSafe, except `execTransaction`, which anyone can call only if Owners of OwnerlessSafe have approved the transaction, and except `fallback` and `receive`, which anyone can call but which do not change state, assuming the fallbackHandler is `address(0)`.	Same as P1.2.
P5.OwnerlessSafe.2	OwnerlessSafe fallbackHandler is `address(0)`, which can only be changed by Modules of OwnerlessSafe and Owners of OwnerlessSafe.	**On-chain — OwnerlessSafe fallbackHandler is `address(0)`:** • Same as P1.3. **FV — OwnerlessSafe fallbackHandler can only be changed by Owners of OwnerlessSafe or Modules of OwnerlessSafe:** • SC150-4 `settingsOnlyChangeThroughAModuleOrOwners` ✅ passes: over every write function, `fallback` included while no fallbackHandler is set (P1.3), the fallbackHandler only changes through an enabled module's `execTransactionFromModule` or `execTransactionFromModuleReturnData`, or through `execTransaction`. • By P1.2, only Modules of OwnerlessSafe can call `execTransactionFromModule` and `execTransactionFromModuleReturnData` (SE150-3), and `execTransaction` only acts once as many Owners of OwnerlessSafe as the threshold have approved (SE150-5, SE150-12). So only Modules of OwnerlessSafe or Owners of OwnerlessSafe can change the fallbackHandler.
P5.OwnerlessSafe.3	`Owners of OwnerlessSafe = Term multisig holders on the OwnerlessSafe`, which can only be changed by Term multisig holders on the OwnerlessSafe and Modules of OwnerlessSafe.	**On-chain — Owners of OwnerlessSafe are these nine Term multisig holders, and all nine must approve each transaction:** • Same as P1.4. **FV — Owners of OwnerlessSafe can only be changed by Term multisig holders on the OwnerlessSafe or Modules of OwnerlessSafe:** • SC150-4 `settingsOnlyChangeThroughAModuleOrOwners` ✅ passes: over every write function, `fallback` included while no fallbackHandler is set (P1.3), the owners and threshold only change through an enabled module's `execTransactionFromModule` or `execTransactionFromModuleReturnData`, or through `execTransaction`. • By P1.2, only Modules of OwnerlessSafe can call `execTransactionFromModule` and `execTransactionFromModuleReturnData` (SE150-3), and `execTransaction` only acts once all nine Owners of OwnerlessSafe, the Term multisig holders (P1.4), have approved (SE150-5, SE150-12). So only Modules of OwnerlessSafe or the Term multisig holders on the OwnerlessSafe can change the owners and threshold.
P5.OwnerlessSafe.4	`Modules of OwnerlessSafe = Delay Modifier`, which can only be changed by Term multisig holders on the OwnerlessSafe and Delay Modifier.	**On-chain — The Modules of OwnerlessSafe list contains only the Delay Modifier:** • Same as P1.5. **FV — Modules of OwnerlessSafe can only be changed by Term multisig holders on the OwnerlessSafe or Delay Modifier:**

#	Statement	Evidence
		• SC150-4 `settingsOnlyChangeThroughAModuleOrOwners` ✅ passes: over every write function, `fallback` included while no fallbackHandler is set (P1.3), the module list only changes through an enabled module's `execTransactionFromModule` or `execTransactionFromModuleReturnData`, or through `execTransaction`. • By P1.2, only Modules of OwnerlessSafe can call `execTransactionFromModule` and `execTransactionFromModuleReturnData` (SE150-3), and `execTransaction` only acts once all nine Owners of OwnerlessSafe, the Term multisig holders (P1.4), have approved (SE150-5, SE150-12). • The OwnerlessSafe's only module is the Delay Modifier (P1.5), so only the Delay Modifier or the Term multisig holders on the OwnerlessSafe can change the module list.
P5.OwnerlessSafe.5	Only Modules of Delay Modifier or Owner of Delay Modifier can interact with Delay Modifier, except `executeNextTx` and `skipExpired`, which anyone can call, but they can only execute or skip transactions queued by Modules of Delay Modifier or Owner of Delay Modifier.	Same as P1.6.
P5.OwnerlessSafe.6	Only Modules of Delay Modifier can queue transactions to Delay Modifier. Owner of Delay Modifier cannot queue transactions to Delay Modifier.	Same as P1.7.
P5.OwnerlessSafe.7	`Modules of Delay Modifier = ProposerSafe`, which cannot be changed by anyone, P5.DelayModifier.	**On-chain — The Modules of Delay Modifier list contains only the ProposerSafe:** • Same as P1.8. **FV — Modules of Delay Modifier can only be changed by Owner of Delay Modifier:** • DM-2 `modulesOnlyChangeThroughOwnerEnableOrDisable` ✅ passes: over every write function, the module list changes only when the caller is the owner (Owner of Delay Modifier). **Already proven — Modules of Delay Modifier cannot be changed by anyone:** • See P5.DelayModifier: by P5.DelayModifier.13, the Term multisig holders on the DelayOwnerSafe cannot change the Modules of Delay Modifier, so nobody can.
P5.OwnerlessSafe.8	Only Modules of ProposerSafe and Owners of ProposerSafe can interact with the ProposerSafe, except `execTransaction`, which anyone can call only if Owners of ProposerSafe have approved the transaction, and except `fallback` and `receive`, which anyone can call but which do not change state, assuming the fallbackHandler is `address(0)`.	Same as P1.9.
P5.OwnerlessSafe.9	ProposerSafe fallbackHandler is `address(0)`, which cannot be changed by anyone, P5.ProposerSafe.	**Already proven — ProposerSafe fallbackHandler is `address(0)`:** • Same as P1.10. **Already proven — ProposerSafe fallbackHandler cannot be changed by anyone:**

#	Statement	Evidence
		• See P5.ProposerSafe: by P5.ProposerSafe.5, the Term multisig holders on the ProposerSafe cannot change the fallbackHandler, so nobody can.
P5.OwnerlessSafe.10	`Modules of ProposerSafe = null`, which cannot be changed by anyone, P5.ProposerSafe.	**Already proven — The Modules of ProposerSafe list is empty:** • Same as P1.12. **Already proven — Modules of ProposerSafe cannot be changed by anyone:** • See P5.ProposerSafe: by P5.ProposerSafe.5, the Term multisig holders on the ProposerSafe cannot change the Modules of ProposerSafe list, so nobody can.
P5.OwnerlessSafe.11	`Owners of ProposerSafe = Term multisig holders on the ProposerSafe`, which can only be changed by Term multisig holders on the ProposerSafe, P5.ProposerSafe.	Same as P5.ProposerSafe.4.
P5.OwnerlessSafe.12	The owners for the OwnerlessSafe can be changed by Term multisig holders on the ProposerSafe and Term multisig holders on the OwnerlessSafe; the fallbackHandler, modules, guard, module guard for the OwnerlessSafe can only be changed by Term multisig holders on the ProposerSafe.	**FV — The Term multisig holders on the OwnerlessSafe can change Owners of OwnerlessSafe through `execTransaction`:** • CL150-21 `guardedOwnersCanStillChangeOtherSettings` ✅ passes on Safe v1.5.0, the OwnerlessSafe's code (P1.2), for each of `addOwnerWithThreshold`, `removeOwner`, `swapOwner` and `changeThreshold`: with ConfigLockGuard installed, an `execTransaction` addressed to the Safe with a call to that function succeeds and the owners or threshold change. • CL150-21 stubs the signature check. By P1.2, `execTransaction` only acts once all nine Owners of OwnerlessSafe, the Term multisig holders (P1.4), have approved (SE150-5, SE150-12), and their approvals pass it (SE150-6). **FV — The Term multisig holders on the ProposerSafe can change the owners, fallbackHandler, modules, guard and module guard of the OwnerlessSafe through the Delay Modifier:** • PO150-5 `proposerApprovalChangesOwnerlessSettings` ✅ passes, for each of `enableModule`, `disableModule`, `addOwnerWithThreshold`, `removeOwner`, `swapOwner`, `changeThreshold`, `setGuard`, `setFallbackHandler` and `setModuleGuard`, in one execution: ◦ An owner of the ProposerSafe (a Term multisig holder, P1.11) approves with `approveHash` the hash of the ProposerSafe's call to the Delay Modifier's `execTransactionFromModule(OwnerlessSafe, 0, <settings call>, Call)`, and anyone submits it through `execTransaction` with that approval as the signature. ◦ The ProposerSafe is the Delay Modifier's module (P1.8), so the Delay Modifier queues the proposal. ◦ Once the cooldown has passed and before the proposal expires, anyone's `executeNextTx` hands it to the Delay Modifier's target, the OwnerlessSafe, which has the Delay Modifier as its module (P1.5). ◦ The OwnerlessSafe's `execTransactionFromModule` makes it call its own settings function, and the setting changes. • PO150-5 sets the ProposerSafe's threshold to 1. By SE150-6 (P1.9), with the real threshold of 5 (P1.11), five owners' approvals take the same path. • PO150-5 assumes no guard on either Safe or on the Delay Modifier. The Delay Modifier's guard is the PauseGuard. DP-18 `checkTransactionAcceptsWhileNotPaused` ✅ passes: while not paused, the PauseGuard accepts any transaction. Its `checkAfterExecution` does nothing (`PauseGuard.sol:123`), so with the PauseGuard not paused, PO150-5's path is unchanged. • The ProposerSafe's guard is ConfigLockGuard (P5.ProposerSafe.5). The ProposerSafe's call is to the Delay Modifier's `execTransactionFromModule`, which is not one of the guard's locked calls. CL150-10 `checkTransactionAcceptsEverythingElse` ✅ passes: whatever `lockedDelay` is, the guard accepts every plain call that is not a locked call. It checks ConfigLockGuard's `checkTransaction` directly, so the Safe version does not matter. Its `checkAfterExecution` does nothing (`ConfigLockGuard.sol:141`), so PO150-5's ProposerSafe step is unchanged. • The OwnerlessSafe's guard is ConfigLockGuard (below), but it is never consulted on this path: Safe v1.5.0 consults its

#	Statement	Evidence
		guard only in `execTransaction` (`Safe.sol:149–204`), not in `execTransactionFromModule` (`ModuleManager.sol:154–163`, which consults only the module guard, `ModuleManager.sol:94–109`). LG150-2 `moduleCanStillChangeEachSetting` ✅ passes: with ConfigLockGuard installed and its `lockedDelay` the Delay Modifier (P5.DelayModifier.13), an `execTransactionFromModule` from the Delay Modifier, as an enabled module, addressed to the Safe with a call to each of `enableModule`, `disableModule`, `addOwnerWithThreshold`, `removeOwner`, `swapOwner`, `changeThreshold`, `setGuard`, `setFallbackHandler` and `setModuleGuard` succeeds and a setting changes. • PO150-5 assumes the deployed wiring: the ProposerSafe is a module of the Delay Modifier (P1.8), and the Delay Modifier is a module of the OwnerlessSafe (P1.5). The Delay Modifier's target is the OwnerlessSafe and its guard is the PauseGuard, read in P1.13. The OwnerlessSafe's module guard, which would check this path's module transaction, must be `address(0)`, also read in P1.13; PO150-5 and LG150-2 run with it unset. • While the PauseGuard is paused, it rejects every `executeNextTx` (`PauseGuard.sol:119`), so the proposal waits until the PauseGuard's admin unpauses it. If `txExpiration` passes first, the proposal expires and has to be proposed again. **FV — The Term multisig holders on the OwnerlessSafe cannot change OwnerlessSafe fallbackHandler, Modules of OwnerlessSafe, OwnerlessSafe guard or OwnerlessSafe module guard through `execTransaction` if OwnerlessSafe guard is ConfigLockGuard, so only the Term multisig holders on the ProposerSafe can change them:** • CL150-2 `checkTransactionRejectsSelfSetGuard`, CL150-3 `checkTransactionRejectsSelfEnableModule`, CL150-4 `checkTransactionRejectsSelfDisableModule`, CL150-5 `checkTransactionRejectsSelfSetFallbackHandler` ✅ pass: called by the Safe, the guard reverts on any transaction to the Safe itself carrying `setGuard`, `enableModule`, `disableModule` or `setFallbackHandler`. • CL150-12 `guardedSafeCannotCallSetGuard`, CL150-13 `guardedSafeCannotCallEnableModule`, CL150-14 `guardedSafeCannotCallDisableModule`, CL150-15 `guardedSafeCannotCallSetFallbackHandler` ✅ pass: with ConfigLockGuard installed, an `execTransaction` addressed to the Safe with any of these calls reverts. • CL150-17 `guardedSelfCallNeverChangesGuard`, CL150-18 `guardedSelfCallNeverChangesModules`, CL150-19 `guardedSelfCallNeverChangesFallbackHandler` ✅ pass: for any calldata and operation, an `execTransaction` addressed to the Safe leaves the guard, every entry of the Modules of OwnerlessSafe list and the fallbackHandler unchanged. • CL150-25 `checkTransactionRejectsSelfSetModuleGuard`, CL150-26 `guardedSafeCannotCallSetModuleGuard`, CL150-27 `guardedSelfCallNeverChangesModuleGuard` ✅ pass: called by the Safe, the guard reverts on any transaction to the Safe itself carrying `setModuleGuard`; with ConfigLockGuard installed, an `execTransaction` addressed to the Safe with that call reverts; and for any calldata and operation, an `execTransaction` addressed to the Safe leaves the module guard unchanged. • CL150-1 `checkTransactionRejectsDelegateCall`, CL150-16 `guardedSafeCannotDelegateCall` ✅ pass: the guard rejects every delegate call, so no delegated code can write the guard, the Modules of OwnerlessSafe list, the fallbackHandler or the module guard directly. • By P1.2, these settings only change when the caller is the Safe itself (SE150-7), so an `execTransaction` to any other address cannot change them either. • These rules run on Safe v1.5.0, the OwnerlessSafe's code (P1.2), and leave `lockedDelay` unconstrained, so they hold whichever modifier ConfigLockGuard was built with. • By P5.OwnerlessSafe.2 and P5.OwnerlessSafe.4, only Owners of OwnerlessSafe or Modules of OwnerlessSafe can change the fallbackHandler and the Modules of OwnerlessSafe list, and SC150-4 (P5.OwnerlessSafe.2) covers the guard and the module guard the same way. The only Module of OwnerlessSafe is the Delay Modifier (P5.OwnerlessSafe.4), which only runs transactions queued by Modules of Delay Modifier (P5.OwnerlessSafe.5, P5.OwnerlessSafe.6). The only Module of Delay Modifier is the ProposerSafe, which cannot be changed by anyone (P5.OwnerlessSafe.7), and only the Term multisig holders on the ProposerSafe can make the ProposerSafe act (P5.OwnerlessSafe.8 to P5.OwnerlessSafe.11). So only the Term multisig holders on the ProposerSafe can change them.

#	Statement	Evidence
		On-chain — OwnerlessSafe guard is ConfigLockGuard: • ⏳ pending deployment. The guard slot (`0x4a204f620c8c5ccdca3fd54d003badd85ba500436a431f0cbda4f558c93c34c8`, `GuardManager.sol:66` in Safe v1.5.0) on the OwnerlessSafe must hold ConfigLockGuard. • Assumes the OwnerlessSafe runs Safe v1.5.0, the version CL150, LG150 and PO150-5 check (`GuardManager.sol:66`).

Proposition 6 — The owners, fallbackHandler, modules, guard for the PauseSafe can be changed only by the Term multisig holders on the PauseSafe. The pauser for the PauseGuard can be changed only by the Term multisig holders on the ProposerSafe. The admin for the PauseGuard cannot be changed by anyone.

PauseSafe

P6.PauseSafe: The owners, fallbackHandler, modules, guard for the PauseSafe can only be changed by Term multisig holders on the PauseSafe.

#	Statement	Evidence
P6.PauseSafe.1	Only Modules of PauseSafe and Owners of PauseSafe can interact with the PauseSafe, except `execTransaction`, which anyone can call only if Owners of PauseSafe have approved the transaction, and except `fallback` and `receive`, which anyone can call but which do not change state, assuming the fallbackHandler is `address(0)`.	Same as P3.Pause.3.
P6.PauseSafe.2	PauseSafe fallbackHandler is `address(0)`, which can only be changed by Owners of PauseSafe and Modules of PauseSafe.	Same as P3.Pause.4. **FV — PauseSafe fallbackHandler can only be changed by Owners of PauseSafe or Modules of PauseSafe:** • SC150-4 `settingsOnlyChangeThroughAModuleOrOwners` ✅ passes: over every write function, `fallback` included while no fallbackHandler is set (P3.Pause.4), the fallbackHandler only changes through an enabled module's `execTransactionFromModule` or `execTransactionFromModuleReturnData`, or through `execTransaction`. • By P6.PauseSafe.1, only Modules of PauseSafe can call `execTransactionFromModule` and `execTransactionFromModuleReturnData` (SE150-3), and `execTransaction` only acts once as many Owners of PauseSafe as the threshold have approved (SE150-5, SE150-12). So only Modules of PauseSafe or Owners of PauseSafe can change the fallbackHandler.
P6.PauseSafe.3	`Modules of PauseSafe = null`, which can only be changed by Owners of PauseSafe because `Modules of PauseSafe = null`.	Same as P3.Pause.6. **FV — Modules of PauseSafe can only be changed by Owners of PauseSafe because `Modules of PauseSafe = null`:** • SC150-4 `settingsOnlyChangeThroughAModuleOrOwners` ✅ passes: over every write function, `fallback` included while no fallbackHandler is set (P3.Pause.4), the module list only changes through an enabled module's `execTransactionFromModule` or `execTransactionFromModuleReturnData`, or through `execTransaction`.

#	Statement	Evidence
		• By P6.PauseSafe.1, only Modules of PauseSafe can call `execTransactionFromModule` and `execTransactionFromModuleReturnData` (SE150-3), and `execTransaction` only acts once as many Owners of PauseSafe as the threshold have approved (SE150-5, SE150-12). • The PauseSafe has no module (P3.Pause.6), so only the Owners of PauseSafe can change the module list.
P6.PauseSafe.4	`Owners of PauseSafe = Term multisig holders on the PauseSafe`, which can only be changed by Term multisig holders on the PauseSafe because `Modules of PauseSafe = null`.	Same as P3.Pause.5. **FV — Owners of PauseSafe can only be changed by Term multisig holders on the PauseSafe because `Modules of PauseSafe = null`:** • SC150-4 `settingsOnlyChangeThroughAModuleOrOwners` ✅ passes: over every write function, `fallback` included while no fallbackHandler is set (P3.Pause.4), the owners and threshold only change through an enabled module's `execTransactionFromModule` or `execTransactionFromModuleReturnData`, or through `execTransaction`. • By P6.PauseSafe.1, only Modules of PauseSafe can call `execTransactionFromModule` and `execTransactionFromModuleReturnData` (SE150-3), and `execTransaction` only acts once two Owners of PauseSafe, the Term multisig holders (P3.Pause.5), have approved (SE150-5, SE150-12). • The PauseSafe has no module (P3.Pause.6), so only the Term multisig holders on the PauseSafe can change the owners and threshold.
P6.PauseSafe.5	PauseSafe guard is `address(0)`, which can only be changed by Owners of PauseSafe because `Modules of PauseSafe = null`.	**On-chain — PauseSafe guard is `address(0)`:** • ⏳ pending deployment. The guard slot (`0x4a204f620c8c5ccdca3fd54d003badd85ba500436a431f0cbda4f558c93c34c8`, `GuardManager.sol:66` in Safe v1.5.0) on the PauseSafe must hold `address(0)`. • Assumes the PauseSafe runs Safe v1.5.0, the version SE150 and SC150 check (`GuardManager.sol:66`). **FV — PauseSafe guard can only be changed by Owners of PauseSafe because `Modules of PauseSafe = null`:** • SC150-4 `settingsOnlyChangeThroughAModuleOrOwners` ✅ passes: over every write function, `fallback` included while no fallbackHandler is set (P3.Pause.4), the guard only changes through an enabled module's `execTransactionFromModule` or `execTransactionFromModuleReturnData`, or through `execTransaction`. • By P6.PauseSafe.1, only Modules of PauseSafe can call `execTransactionFromModule` and `execTransactionFromModuleReturnData` (SE150-3), and `execTransaction` only acts once as many Owners of PauseSafe as the threshold have approved (SE150-5, SE150-12). • The PauseSafe has no module (P3.Pause.6), so only the Owners of PauseSafe can change the guard.
P6.PauseSafe.6	The owners, fallbackHandler, modules, guard for the PauseSafe can be changed by Term multisig holders on the PauseSafe.	**FV — The Term multisig holders on the PauseSafe can change the owners, fallbackHandler, modules and guard for the PauseSafe through `execTransaction`:** • SC150-3 `ownersCanChangeSettings` ✅ passes: for each of `addOwnerWithThreshold`, `removeOwner`, `swapOwner`, `changeThreshold`, `enableModule`, `disableModule`, `setGuard` and `setFallbackHandler`, an owner's (Owners of Safe) `execTransaction`, addressed to the Safe with a call to that function, succeeds and changes a setting once the threshold is passed. • SE150-6 `ownersCanMakeTheSafeAct` ✅ passes: once the threshold is passed, `execTransaction` succeeds and the Safe acts, for any threshold and any number of owners. The PauseSafe runs Safe v1.5.0 (P3.Pause.5), and its owners are the Term multisig holders on the PauseSafe (P3.Pause.5). • The PauseSafe has no guard (P6.PauseSafe.5), so nothing rejects the owners' `execTransaction`.

PauseGuard

P6.PauseGuard: The pauser for the PauseGuard can only be changed by Term multisig holders on the ProposerSafe; the admin for the PauseGuard cannot be changed by anyone.

#	Statement	Evidence
P6.PauseGuard.1	Only the PauseGuard pauser and the PauseGuard admin can interact with the PauseGuard, except `checkTransaction` and `checkAfterExecution`, which anyone can call but which do not change state.	Same as P3.1.
P6.PauseGuard.2	`PauseGuard admin = ProposerSafe`, which cannot be changed by anyone.	Same as P3.Admin.2. **FV — Nobody can change the PauseGuard admin:** • DP-14 `adminNeverChanges` ✅ passes: over every write function of the Delay Modifier and the PauseGuard, the admin never changes. • `admin` is `immutable` (`PauseGuard.sol:17`), set once in the constructor, so no function can write it.
P6.PauseGuard.3	`PauseGuard pauser = PauseSafe`, which can only be changed by Term multisig holders on the ProposerSafe.	Same as P3.Pause.2. **FV — The PauseGuard pauser can only be changed by Term multisig holders on the ProposerSafe:** • DP-13 `pauserOnlyChangesThroughSetPauser` ✅ passes: over every write function of both contracts, the pauser only changes through `setPauser`. • DP-10 `setPauserRevertsForAnyoneButTheAdmin` ✅ passes: anyone who is not the admin is rejected. • The PauseGuard admin is the ProposerSafe (P6.PauseGuard.2), and only the Term multisig holders on the ProposerSafe can make it act (P5.ProposerSafe.1, P5.ProposerSafe.3, P5.ProposerSafe.4). So only they can change the pauser.
P6.PauseGuard.4	The pauser for the PauseGuard can be changed by Term multisig holders on the ProposerSafe; the admin for the PauseGuard cannot be changed by Term multisig holders on the ProposerSafe, so it cannot be changed by anyone.	**FV — The Term multisig holders on the ProposerSafe can change the PauseGuard pauser:** • DP-11 `setPauserSetsThePauser` ✅ passes: for any nonzero account, the admin's `setPauser(account)` succeeds, `pauser()` then returns that account, and the previous pauser no longer holds the slot. • SE150-6 `ownersCanMakeTheSafeAct` ✅ passes: once the threshold is passed, `execTransaction` succeeds and the Safe acts, for any threshold and any number of owners. So the ProposerSafe's owners (P3.Admin.5) can have it call `setPauser`. • The ProposerSafe's guard is ConfigLockGuard (P5.ProposerSafe.5). Its call to `setPauser` on the PauseGuard is a plain call to neither the ProposerSafe nor its locked Delay Modifier, so it is not a locked call. CL150-10 `checkTransactionAcceptsEverythingElse` ✅ passes: whatever `lockedDelay` is, the guard accepts every plain call that is not a locked call. It checks ConfigLockGuard's `checkTransaction` directly, so the Safe version does not matter. Its `checkAfterExecution` does nothing (`ConfigLockGuard.sol:141`), so the call is unchanged. **FV — The Term multisig holders on the ProposerSafe cannot change the PauseGuard admin, so nobody can:** • Same as P6.PauseGuard.2: the admin never changes, whoever the caller is, the ProposerSafe included.

4. Appendix

4.1 Formal Verification

All Certora specs and confs for this document: `packages/evm/certora/2.1.0`.

Each rule cited in Section 3 is named by a prefix and a number, such as SE150-5 or DM-13. The table below lists each prefix, what its rules cover, which of its rules Section 3 uses, the spec that defines them and the conf that runs them. A prefix whose rules are spread over more than one spec has one row per spec.

Prefix	What it covers	Rules used in Section 3	Spec	Conf
SE150	Safe v1.5.0 entry points and who can make the Safe act	SE150-1, SE150-3 to SE150-9, SE150-11, SE150-12	`SafeV150/executionPaths.spec`	`Safe-v150-executionPaths.conf`
	Safe v1.5.0 `fallback` and `receive` change no state	SE150-21 to SE150-26	`SafeV150/noStateChange.spec`	`Safe-v150-noStateChange.conf`
SC150	Safe v1.5.0 settings change only through a module or the owners	SC150-1 to SC150-4	`SafeV150/selfCalls.spec`	`Safe-v150-selfCalls.conf`
CL150	ConfigLockGuard on a Safe v1.5.0 Safe: what owners cannot change on the Safe, its module guard included, and its locked modifier, what owners can still do, and the module path it does not check	CL150-1 to CL150-10, CL150-12 to CL150-19, CL150-21 to CL150-27	`SafeV150/configLockGuard.spec`	`Safe-v150-configLockGuard.conf`
LG150	ConfigLockGuard on the OwnerlessSafe: the module path	LG150-2, LG150-3	`SafeV150/ownerlessSafeGuard.spec`	`Safe-v150-ownerlessSafeGuard.conf`
OD150	ConfigLockGuard locking the Delay Modifier: no unlocked call changes its owner, target, modules, guard, txCooldown or txExpiration, and the DelayOwnerSafe can still veto	OD150-1, OD150-2, OD150-4, OD150-5	`Delay/SafeV150/delayOwnerGuard.spec`	`Delay-Safe-v150-delayOwnerGuard.conf`
DM	Delay Modifier access control and module list	DM-1, DM-2, DM-4 to DM-8, DM-11 to DM-13, DM-15 to DM-20	`Delay/moduleIntegrity.spec`	`Delay-moduleIntegrity.conf`
DQ	Delay Modifier queue provenance	DQ-2 to DQ-6	`Delay/queueProvenance.spec`	`Delay-queueProvenance.conf`
DE150	DEVOPS_ROLE methods end to end	DE150-1, DE150-2	`Delay/SafeV150/devopsEndToEnd.spec`	`Delay-Safe-v150-devopsEndToEnd.conf`
DV150	DelayOwnerSafe owners veto directly	DV150-1	`Delay/SafeV150/vetoSignedPath.spec`	`Delay-Safe-v150-vetoSignedPath.conf`
DP	PauseGuard, and the Delay Modifier's veto window and vetoes	DP-3 to DP-18, DP-24 to DP-33	`Delay/pauseGuardSufficient.spec`	`Delay-pauseGuardSufficient.conf`

Prefix	What it covers	Rules used in Section 3	Spec	Conf
PO150	ProposerSafe → Delay Modifier → OwnerlessSafe settings	PO150-5	`Delay/SafeV150/proposerToOwnerlessSettings.spec`	`Delay-Safe-v150-proposerToOwnerlessSettings.conf`
RI	Roles Modifier configuration integrity	RI-1, RI-2, RI-5 to RI-16, RI-19, RI-20, RI-22 to RI-24	`Roles/configIntegrity.spec`	`Roles-configIntegrity.conf`
RG	Governor limited to `Delay.setTxNonce`	RG-2 to RG-5	`Roles/setTxNonceGuardAndRoleConfig.spec`	`Roles-setTxNonceGuardAndRoleConfig.conf`
GP	Governor proposal threshold	GP-1	`Governor/proposalThreshold.spec`	`Governor-proposalThreshold.conf`
GS	Governor entry points and settings	GS-1 to GS-18	`Governor/settings.spec`	`Governor-settings.conf`
GC	Governor vote counting	GC-1 to GC-3, GC-5	`Governor/voting.spec`	`Governor-voting.conf`
	Only votes move a tally	GC-4	`Governor/tallies.spec`	`Governor-tallies.conf`
GV	Governor veto	GV-1, GV-2, GV-4	`Governor/veto.spec`	`Governor-veto.conf`
	The veto lands on the DelayOwnerSafe (Safe v1.5.0)	GV-3	`SafeV150/vetoLandsOnDelay.spec`	`Safe-v150-vetoLandsOnDelay.conf`
GE	Veto end to end	GE-1	`Governor/vetoEndToEnd.spec`	`Governor-vetoEndToEnd.conf`

4.2 On-chain

All contracts are on Ethereum mainnet. Each address links to its Etherscan page. Contracts marked to be deployed are part of the proposed 2.1.0 deployment and have no address yet.

Safes

Contract	Address
OwnerlessSafe	to be deployed
ProposerSafe	to be deployed
DelayOwnerSafe	to be deployed
PauseSafe	to be deployed
Safe v1.5.0 mastercopy	`0xFf51A5898e281Db6DfC7855790607438dF2ca44b`

Modifiers

Contract	Address
Delay Modifier	to be deployed
Roles Modifier	to be deployed

Guards

Contract	Address
PauseGuard	to be deployed
SetTxNonceGuard	to be deployed
ConfigLockGuard	to be deployed

Governance

Contract	Address
Governor	to be deployed
TERM token	0xC3d21f79C3120A4fFda7A535f8005a7c297799bF

Term Protocol contracts

Contract	Address
TermController	to be deployed
TermEventEmitter	to be deployed
TermPriceConsumerV3	to be deployed
TermRepoDeployerFactory	to be deployed
TermInitializer	to be deployed
TermDiamond	to be deployed